

Kiss ISA Commands

License Notice Copyright © 2026 Wolven LLC. This document is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). See [docs/Licenses/CC-BY-4.0.txt](#) and <https://creativecommons.org/licenses/by/4.0/>.

1. Introduction

1.1 Purpose

This document defines the commands and command-format structures of the Kiss Instruction Set Architecture (ISA). Its purpose is to present the Kiss command families, command-bit usage, and related encoding conventions in a clear and organized reference format.

This document focuses on command organization, bit usage, and command-family formats. It is intended to support understanding, implementation, and use of the Kiss ISA command set.

1.2 Scope

This document covers the command families of the Kiss ISA, the layout of their command-related fields, and the conventions needed to interpret those layouts. It describes shared instruction-format rules, family-specific command-bit usage, and related command encoding behavior.

This document focuses on the current structure and meaning of Kiss ISA commands. Additional implementation details, examples, and related architectural topics may be included where they help clarify command behavior.

2. Conventions

2.1 Naming and datatype notation

Kiss command names are described in terms of command families and command variants. In many cases, a single underlying ISA command corresponds to multiple Kiss Assembly Language command forms. Where datatype-specific command forms exist, datatype notation is shown explicitly as part of the command name.

The Kiss documentation uses datatype-prefixed naming forms such as **B.Add**, **I.Add**, and **D.Add** where appropriate. In these forms:

- **B** denotes Binary
- **I** denotes Integer
- **D** denotes Decimal

Additional datatype forms may be introduced where required by the architecture.

When needed, command variants may also indicate direction, carry or borrow behavior, status-flag behavior, or other family-specific options. These meanings are defined in the relevant family sections of this document.

2.2 Operand notation

This document uses compact operand-role notation when describing command formats. The notation identifies both the kind of operand and the role the operand plays within a command.

The most common operand markers are:

- **R** — Register ID
- **sR** — Source register; the register is read but not modified
- **dR** — Destination or target register; the register is written
- **sdR** — Source-and-destination register; the register is read, then written (in-place modification)
- **iR** — Immediate or register; the operand may be either an immediate value or a register ID (CMB-3 selects)
- **asR** — Source-address register; the register holds a memory address and the memory at that address is read (Load family)
- **adR** — Destination-address register; the register holds a memory address and the memory at that address is written (Stor family)
- **o** prefix — Optional operand
- **?** prefix — Uncertain or not yet fully defined usage

Examples:

- **R1** means the register ID identified in operand 1
- **sR1** means operand 1 is a source register (read, not modified)
- **iR1** means operand 1 may be an immediate value or a register
- **dR5** means operand 5 is the destination register (primary destination is always operand 5 at the ISA level)
- **sdR5** means operand 5 is read then written (in-place operation)
- **asR2** means operand 2 holds a memory address being read from
- **adR3** means operand 3 holds a memory address being written to

- **o** means this is an optional operand register
- **osR2** means operand 2 is an optional source register
- **odR4** means operand 4 is an optional destination register

These operand-role forms are used as a compact notation within command descriptions. Their exact meaning depends on the command family being described. See [docs/isa/kiss-isa-operand-notation.md](#) for the complete reference, including per-family operand patterns derived from the **KSISAOPr** table.

2.3 Reading the format grids

Command-format grids in this document show the structure of a Kiss instruction or command-family layout. They identify which bit fields are relevant to a particular family and how those fields are interpreted.

A format grid may show:

- fields that are common across Kiss instructions
- fields whose meanings vary by command family
- grouped bits that together represent a datatype, modifier, or option set
- operand fields and any special usage rules associated with them

Not every field in a format grid requires the same level of attention. In many cases, only the command-related fields or family-specific option bits are central to the discussion. Supporting notes below each grid should explain any special interpretations, restrictions, or exceptions that apply.

3. Common instruction structure

3.1 Standard instruction layout

Kiss instructions use a common structural layout. This layout defines the major instruction sections used throughout the ISA and provides the framework within which individual command families assign meaning to the Command Modifier Bits.

The standard Kiss instruction format contains the following sections:

- Conditional Execution Bits (CEB) — 5 bits
- Command — 12 bits
- Command Modifier Bits (CMB) — 8 bits
- Operand 1 — 15 bits

- Operand 2 — 15 bits
- Operand 3 — 15 bits
- Operand 4 — 15 bits
- Operand 5 — 15 bits

Within the Conditional Execution Bits section, the five bits are identified individually as:

- CEB-1 — 1 bit
- CEB-2 — 1 bit
- CEB-3 — 1 bit
- CEB-4 — 1 bit
- CEB-5 — 1 bit

Within the Command Modifier Bits section, the eight bits are identified individually as:

- CMB-1 — 1 bit
- CMB-2 — 1 bit
- CMB-3 — 1 bit
- CMB-4 — 1 bit
- CMB-5 — 1 bit
- CMB-6 — 1 bit
- CMB-7 — 1 bit
- CMB-8 — 1 bit

The standard instruction layout should be shown with a general format grid that identifies these sections and their bit widths without assigning any family-specific meaning to the CMB bits.

A second grid should show the current **SetImm** payload form. In that form, the instruction still uses the common Kiss layout, but the operands are interpreted specially:

- Conditional Execution Bits (CEB) — 5 bits
- Command — 12 bits
- Command Modifier Bits (CMB) — 8 bits
- Operand 1 — **destination register** (special-form exception to destination-last)
- Operands 2-5 — **50-bit payload** (right-justified in the 60-bit Opr 2-5 span; bits 41-50 are

unused padding)

As with the standard layout, the CEB section and the CMB section in the **SetImm** form should be shown as individual one-bit fields.

These two grids establish the common structural layouts used by Kiss instructions. Later family sections can then focus only on the meanings that are specific to that family rather than repeating the full instruction structure.

3.2 Conditional execution bits

The five-bit Conditional Execution Bits (CEB) field controls whether an instruction is unconditional or conditional, and identifies which active nesting level a conditional instruction belongs to.

- **CEB-1:** conditional-execution enable. **0** = unconditional, **1** = conditional.
- **CEB-2 through CEB-5:** the encoded condition value. For **If*** nesting, this is a one-hot bitmask identifying the active nesting level (CEB-2 = first level, CEB-3 = second, CEB-4 = third, CEB-5 = fourth).

CEB Value	Meaning
00000	Unconditional / top-level
11000	First nested If level
10100	Second nested If level
10010	Third nested If level
10001	Fourth nested If level

3.2.1 Execution mechanism

Each thread maintains three CEB-related state elements:

- **Main CEB** — the currently active CEB value. An instruction executes only when its CEB field matches Main CEB.
- **Shadow CEB** — saved Main CEB value, restored when the current **If** block ends.
- **Depth counter** — current **If*** nesting depth (0 = top-level).

If* instructions evaluate a condition. When **true**, they copy Main → Shadow, increment Depth, and load the next nested CEB value into Main. When **false**, the nested level is never activated and the block is skipped entirely.

EndIf matches its instruction CEB against Shadow rather than Main. When the match succeeds it restores Shadow → Main and decrements Depth, closing exactly one nesting level.

If an instruction's CEB does not match Main, the instruction is suppressed — treated as a NOP without side effects.

3.2.2 Nesting depth

The current implementation supports a maximum of **4** active nesting levels. Depth is tracked explicitly rather than inferred from the CEB bit pattern, so **If** / **EndIf** matching is unambiguous.

There is no separate **Else** instruction at the ISA level — **Else** is an assembler construct that expands into **If*** / **EndIf** pairs that manipulate CEB state directly.

3.2.3 Interaction with Jump and Call

Jump instructions can leave conditional blocks without going through the matching **EndIfs**. They include a 3-bit “CEB clear count” in CMB bits 6–8 that tells the hardware how many active nesting levels to unwind when the jump is taken. This keeps Main / Shadow / Depth consistent regardless of how many nesting boundaries the jump crosses.

Call instructions take a different approach: they save the full CEB state (along with R1–R4, the Status Flag register **SF**, and the special-register set) to the protected shadow call stack, then start the called routine in unconditional mode (Main = **00000**, Depth = 0). R5 is **not** saved by hardware because it is the parameter-passing bank. **Retrn** restores all saved state from the shadow call stack.

3.3 Command and Command Modifier Bits

The 12-bit Command field selects the base operation. The eight-bit Command Modifier Bits (CMB) field modifies the behavior of that operation without changing its command code. A single ISA command like **Add** covers many behavioral variations through its CMB encoding.

3.3.1 Bit numbering

CMB bits are referenced two ways throughout the documentation. The CMB-relative numbering (CMB-1 through CMB-8) is used in family descriptions; the absolute instruction-bit numbering is used when discussing the encoded instruction word.

CMB-Relative	Absolute (Instruction)
CMB-1	bit 18
CMB-2	bit 19
CMB-3	bit 20
CMB-4	bit 21
CMB-5	bit 22
CMB-6	bit 23
CMB-7	bit 24
CMB-8	bit 25

3.3.2 Global conventions

Three CMB assignments are consistent across most command families:

CMB 1-2 — Data type. For typed commands, these bits select which ALU processes the instruction:

CMB 1-2	Type	ALU
00	Untyped / bit-processing / not used	—
01	Binary	Binary ALU (adder)
10	Integer	Integer ALU (subtractor)
11	Decimal	Decimal ALU (matrix lookup)

CMB-6 — Set Status Flags. For families that optionally update arithmetic or compare flags:

- 0 = do not update flags
- 1 = update flags after the operation

CMB 7-8 — Immediate data type. When CMB-3 indicates that an operand is an immediate value, CMB 7-8 encode the immediate's data type using the same encoding as CMB 1-2.

3.3.3 Common patterns

Most command families use one of three CMB layout patterns:

Pattern A — Typed compute (arithmetic, logic, shift / rotate, compare):

- CMB 1-2 = data type
- CMB 3-5 = family-specific modifiers (immediate indicator, direction, carry, saturation, etc.)
- CMB-6 = Set Status Flags
- CMB 7-8 = immediate data type (when applicable)

Pattern B — Data movement (Load / Stor, Read / Write, Copy / Move):

- CMB 1-2 = datatype / payload subtype where the family defines one, otherwise unused
- CMB 3-4 = addressing mode or direction
- CMB 5-8 = family-specific or unused

Pattern C — Simple families (control-flow utilities, register utilities):

- CMB 1-8 = all zero (no modifiers)

Per-family CMB layouts are documented in each family's section below.

3.3.4 Unused bits

Any CMB bit not defined for a given command must be set to 0. The hardware may trap or behave unpredictably on nonzero reserved bits.

3.4 Immediate and datatype conventions

Some Kiss commands support operand forms in which a value may be supplied either through a register or as an immediate value. In command descriptions, this is indicated by operand-role notation such as **iR**.

When a command family allows an immediate value in operand 1, CMB-7 and CMB-8 may be used to encode the datatype of that immediate value. In those cases, the datatype encoding is:

- **01** = Binary
- **10** = Integer
- **11** = Decimal

Datatype may also appear in other ways depending on the command family. In some cases it is reflected in the Kiss Assembly Language command form. In other cases it is encoded through the Command Modifier Bits or through special instruction-format rules.

The primary general datatypes currently used are:

- Binary
- Integer
- Decimal

Some commands also use Text or other special forms where required by their instruction encoding.

This section defines only the common immediate and datatype conventions. Any command family or special command group that interprets datatype differently should define that behavior explicitly in its own section.

3.5 Condition codes

3.5.1 Overview

Several Kiss command families support conditional variants that execute only when a specified condition is true at the time of execution. The Jump, Call, and If command families all share a common set of condition suffixes. These suffixes are appended to the base command name in Kiss Assembly Language to produce the conditional form of that command.

The condition suffix identifies which processor condition is tested. The underlying condition evaluation is separate from the Conditional Execution Bits (CEB) mechanism described in section 3.2. CEB controls whether an instruction executes at all based on the active conditional-execution state. Condition suffixes within Jump, Call, and If commands control the behavior of the command itself — for example, whether a jump is taken or a call is made.

3.5.2 Shared condition suffixes

The following 30 condition suffixes are shared across the Jump, Call, and If command families. Each suffix has a fixed meaning regardless of the command family in which it appears.

Suffix	Meaning
Z	Zero
NZ	Not Zero
Blw	Below
BlwE	Below or Equal
LT	Less Than
LE	Less Than or Equal
EQ	Equal
NE	Not Equal
GE	Greater Than or Equal
GT	Greater Than
AbvE	Above or Equal
Abv	Above
Pos	Positive
NPos	Not Positive
Neg	Negative
NNeg	Not Negative
EvN	Even
Odd	Odd
Ovr	Overflow
NOvr	Not Overflow
Und	Underflow
NUnd	Not Underflow
Brw	Borrow
NBrw	Not Borrow
Cry	Carry
NCry	Not Carry
THi	High-end truncation
NTHi	No high-end truncation
TLo	Low-end truncation
NTLo	No low-end truncation

3.5.3 Notes

The condition suffixes listed above represent the complete shared set. Individual command families may define additional condition forms that are specific to that family. In particular, the If family supports an additional **IfIRQ** form that tests for an active interrupt request. **IRQ** does not

appear in the shared condition-suffix table because it is not available as a suffix for Jump or Call commands.

When a condition suffix is applied to a command, the resulting assembler form combines the base command name with the suffix. For example, **JmpZ**, **CallNE**, and **IfGT** are all valid conditional forms constructed from a base command and a shared condition suffix.

4. Control and conditional-execution commands

4.1 Overview

Control and conditional-execution commands manage processor execution behavior rather than data. They are distinct from jump and call commands: jumps and calls transfer execution to another instruction location, while this family affects how execution proceeds at the current site, manages processor state, or opens and closes conditional-execution blocks.

The family splits into two groups:

- **Execution control commands** (section 4.2): **NOP**, **Brk**, **Retrn**, **ClrCF**.
- **Conditional block commands** (section 4.3): the **If*** family (27 variants including **IfIRQ**) and **EndIf**, which open and close conditional-execution blocks.

Every command in this family uses no CMB bits (all eight CMB bits are 0) and most use no operands. Each command is identified entirely by its Command field. Command-specific details are given below.

4.2 Execution control commands

Four commands make up this subgroup. All use no operands and no CMB bits; each is identified by its Command opcode alone.

Command	Opcode (decimal)	Role
NOP	10	No-operation — pipeline placeholder
Brk	11	Breakpoint — debugger / simulator trap
Retrn	12	Return from subroutine
ClrCF	14	Clear conditional-execution state

4.2.1 NOP — No operation

NOP consumes a pipeline slot but makes no architectural change. It is used for padding, scheduling, and as a placeholder when a slot must be present but should have no effect.

- **Operands:** none.
- **CMB:** all zero.
- **Effect on state:** none. No flags, no registers, no memory, no CEB state is touched.

4.2.2 Brk — Breakpoint

Brk signals a breakpoint to the simulator or attached debugger. It is the architectural stop-for-tooling instruction and is not a general reset or program-termination mechanism.

- **Operands:** none.
- **CMB:** all zero.
- **Effect on state:** hardware-defined trap. Execution of the thread is suspended until the debugger / tool dismisses the breakpoint; no architectural register is modified by **Brk** itself.

4.2.3 Retrn — Return from subroutine

Retrn completes a subroutine and restores the caller's protected machine state. It is the mirror of **Call***.

- **Operands:** none.
- **CMB:** all zero.
- **Effect on state:** pops one entry from the thread's shadow call stack and restores the saved state (R1–R4, Status Flag register **SF**, CEB state, and the special-register set). R5 is not restored — it is the parameter / return-value bank and remains under caller / callee control. Execution resumes at the instruction after the originating **Call***.

Retrn does not carry a target operand: the return location is recovered from the saved state, not from an explicit address.

4.2.4 ClrCF — Clear conditional-execution state

ClrCF resets the active conditional-execution state on the current thread. It forces Main CEB, Shadow CEB, and the Depth counter to their initial values (Main = 00000, Shadow = 00000, Depth = 0).

- **Operands:** none.
- **CMB:** all zero.
- **Effect on state:** CEB state only. No data registers, flags, or memory are affected.

ClrCF is the explicit escape hatch for conditional-execution nesting. It is most useful in error handlers or recovery code where the normal **If*** / **EndIf** unwinding chain cannot be relied on. Normal code should prefer structured **EndIf** nesting or the CEB clear count in Jump instructions.

4.3 Conditional block commands

The **If*** family opens structured conditional-execution blocks; **EndIf** closes them. Each **If*** variant tests a specific processor condition. When the condition is true, the command pushes the current CEB state and activates the next nested conditional level. When the condition is false, the nested

level is not activated and the block is skipped. **EndIf** closes exactly one active nesting level.

The full mechanism (Main CEB, Shadow CEB, Depth counter, state transitions) is documented in section 3.2.

4.3.1 Command list

Command	Opcode	Condition tested
IfIRQ	1630	Interrupt-request pending (hardware flag)
IfZ	1631	Zero flag set
IfNZ	1632	Zero flag clear
IfBlw	1633	Below (unsigned)
IfBlwE	1634	Below or equal (unsigned)
IfLT	1635	Less than (signed)
IfLE	1636	Less than or equal (signed)
IfEQ	1637	Equal
IfNE	1638	Not equal
IfGE	1639	Greater than or equal (signed)
IfGT	1640	Greater than (signed)
IfAbvE	1641	Above or equal (unsigned)
IfAbv	1642	Above (unsigned)
IfPos	1643	Result was positive
IfNPos	1644	Result was not positive
IfNeg	1645	Result was negative
IfNNeg	1646	Result was not negative
IfEvn	1647	Result is even
IfNEvn	1648	Result is not even
IfOdd	1649	Result is odd
IfNOdd	1650	Result is not odd
IfOvr	1651	Overflow flag set
IfNOvr	1652	Overflow flag clear
IfUnd	1653	Underflow flag set
IfNUnd	1654	Underflow flag clear
IfBrw	1655	Borrow flag set
IfNBrw	1656	Borrow flag clear
IfCry	1657	Carry flag set
IfNCry	1658	Carry flag clear
EndIf	1659	Closes one active If level
IfTHi	1660	High-end truncation flag set
IfNTHi	1661	High-end truncation flag clear

Command	Opcode	Condition tested
IfTLo	1662	Low-end truncation flag set
IfNTLo	1663	Low-end truncation flag clear

Total: 33 **If*** variants plus **EndIf**. **IfIRQ** is specific to the **If** family — it is not part of the shared condition-suffix set used by Jump and Call, because interrupt-pending is a processor-state flag rather than an arithmetic result.

4.3.2 Command Modifier Bit usage

All **If*** variants and **EndIf** have all eight CMB bits set to 0. The condition is encoded in the Command field (distinct opcode per variant), not in the CMB.

4.3.3 Operand usage

If* variants and **EndIf** take no operands. Operand 1 through Operand 5 are unused and must be 0.

4.3.4 Assembler forms

The assembler command names match the ISA command names exactly. There is no separate lowering step for the **If*** family.

The assembler keyword **Else** is **not** an ISA command. It is an assembler construct that expands into an **EndIf** closing the current **If** block plus a new, complementary **If*** opening a new block with the opposite condition. The underlying instruction stream contains only standard **If*** / **EndIf** pairs.

4.3.5 Notes

- The maximum nesting depth is **4**. Attempting to open a fifth nested **If*** exceeds the hardware's Depth counter range; the behavior of exceeding this limit is implementation-defined and should be treated as a programming error.
- **If*** evaluates the condition at execution time, using the flags set by the most recent flag-affecting operation (Compare, Test, arithmetic with **Set**, etc.).
- Compare-group flags (EQ, GT, GE, LT, LE, Abv, AbvE, Blw, BlwE), arithmetic-group flags (OVR, UND, WRP, CRY, BRW, INF, ANY_CARRY), and result-property-group flags (Z, Pos, Neg, EVN, ODD) live in three independent groups. An arithmetic operation after a **Compare** does not overwrite the compare flags; **Test** sets Result-property + Compare without disturbing the Arithmetic group.
- **ClrCF** (section 4.2.4) resets CEB state without requiring balanced **EndIfs** and is the recommended recovery mechanism when normal nesting cannot be unwound.

5. Jump commands

5.1 Overview

Jump commands transfer execution to a target instruction location. The target may be specified as an absolute address or as a PC-relative offset. Jump commands do not save a return address and are not intended for call/return behavior. Call commands (section 6) serve that purpose.

The Jump family provides three base assembler forms:

- **Jmp** — jump to an absolute target address
- **FJmp** — jump forward relative to the current PC
- **BJmp** — jump backward relative to the current PC

Any of the 30 shared condition suffixes defined in section 3.5 may be appended to the end of any base jump form to produce a conditional jump. The direction prefix (F/B) precedes the base **Jmp** mnemonic, and the condition suffix follows it. For example, **JmpZ**, **FJmpNZ**, and **BJmpLE** are all valid conditional jump forms. A conditional jump transfers execution only when the specified condition is true; otherwise the instruction has no effect.

The target operand is an **iR** operand (operand 1), meaning the target value may be supplied as an immediate value or as a register containing the value. There is no hardwired special-purpose jump register.

For PC-relative forms (**FJmp** and **BJmp**), the reference point is the PC (Program Counter) value of the jump instruction itself. **FJmp** computes the target as PC + operand 1; **BJmp** computes the target as PC – operand 1.

5.2 Command Modifier Bit usage

The Command Modifier Bits for Jump commands are organized as follows. For the general instruction layout, refer to section 3.1.

- **CMB 1-2:** Immediate-value data type. These bits encode the datatype of the immediate value when CMB-3 indicates an immediate operand. The encoding follows the convention described in section 3.4.
- **CMB-3:** Operand-1 source kind. When 0, operand 1 contains a register ID. When 1, operand 1 contains an immediate value (and CMB 1-2 identify its datatype).
- **CMB-4:** PC-relative mode. When 0, the target is an absolute address. When 1, the target is PC-relative.
- **CMB-5:** Backward-relative selector. This bit is meaningful only when CMB-4 is 1. When 0, the offset is applied forward (PC + operand 1). When 1, the offset is applied backward (PC – operand 1). When CMB-4 is 0, CMB-5 has no effect.

- **CMB 6–8:** CEB clear count. These three bits encode how many active conditional-execution levels to clear if the jump executes. This mechanism supports jumps that exit one or more levels of conditional execution without passing through the corresponding **EndIf** boundaries.

The three base assembler forms correspond to specific CMB-4 and CMB-5 settings:

- **Jmp** — CMB-4 = 0, CMB-5 = 0 (absolute)
- **FJmp** — CMB-4 = 1, CMB-5 = 0 (forward-relative)
- **BJmp** — CMB-4 = 1, CMB-5 = 1 (backward-relative)

5.3 Operand usage

Jump commands use a single operand:

- **Operand 1 (iR1):** The jump target. This is an **iR** operand — it may be an immediate value or a register ID, as determined by CMB-3. When the operand is an absolute address (CMB-4 = 0), operand 1 references the target address directly. When the operand is a PC-relative offset (CMB-4 = 1), operand 1 references the magnitude of the offset.

Operands 2 through 5 are not used by Jump commands.

5.4 Notes

The CEB clear count in CMB 6–8 allows the compiler to encode structural information about conditional-execution nesting. When a jump transfers execution past one or more **EndIf** boundaries, the processor must clear the corresponding active conditional-execution levels so that subsequent instructions are not incorrectly suppressed. CMB 6–8 tell the processor how many CEB levels to clear when the jump executes, avoiding the need for the processor to scan forward or track nesting depth at runtime.

This mechanism is specific to Jump commands. Call commands handle CEB state differently (see section 6).

6. Call commands

6.1 Overview

Call commands transfer execution to a target instruction location for the purpose of invoking a subroutine or function. Unlike Jump commands, Call commands are specifically designed for call/return behavior. When a Call executes, protected call-frame state is saved on the shadow call stack so that a subsequent **Retrn** can resume execution after the call site with the correct saved state restored. Return mechanics are documented separately with the **Retrn** command in the control and conditional-execution family (section 4).

The Call family provides three base assembler forms:

- **Call** — call to an absolute target address
- **FCall** — call forward relative to the current PC
- **BCall** — call backward relative to the current PC

Any of the 30 shared condition suffixes defined in section 3.5 may be appended to the end of any base call form to produce a conditional call. The direction prefix (F/B) precedes the base **Call** mnemonic, and the condition suffix follows it. For example, **CallZ**, **FCallNZ**, and **BCallLE** are all valid conditional call forms. A conditional call transfers execution only when the specified condition is true; otherwise the instruction has no effect.

The target operand is an **iR** operand (operand 1), meaning the target value may be supplied as an immediate value or as a register containing the value. There is no hardwired special-purpose call register.

For PC-relative forms (**FCall** and **BCall**), the reference point is the PC value of the call instruction itself. **FCall** computes the target as PC + operand 1; **BCall** computes the target as PC – operand 1.

6.2 Command Modifier Bit usage

The Command Modifier Bits for Call commands are organized as follows. For the general instruction layout, refer to section 3.1.

- **CMB 1-2:** Immediate-value data type. These bits encode the datatype of the immediate value when CMB-3 indicates an immediate operand. The encoding follows the convention described in section 3.4.
- **CMB-3:** Operand-1 source kind. When 0, operand 1 contains a register ID. When 1, operand 1 contains an immediate value (and CMB 1-2 identify its datatype).
- **CMB-4:** PC-relative mode. When 0, the target is an absolute address. When 1, the target is PC-relative.
- **CMB-5:** Backward-relative selector. This bit is meaningful only when CMB-4 is 1. When 0, the offset is applied forward (PC + operand 1). When 1, the offset is applied backward (PC – operand 1). When CMB-4 is 0, CMB-5 has no effect.
- **CMB 6-8:** Unused. These bits must be 0 for Call commands.

The three base assembler forms correspond to specific CMB-4 and CMB-5 settings:

- **Call** — CMB-4 = 0, CMB-5 = 0 (absolute)
- **FCall** — CMB-4 = 1, CMB-5 = 0 (forward-relative)

- **BCall** — CMB-4 = 1, CMB-5 = 1 (backward-relative)

6.3 Operand usage

Call commands use a single operand:

- **Operand 1 (iR1):** The call target. This is an **iR** operand — it may be an immediate value or a register ID, as determined by CMB-3. When the operand is an absolute address (CMB-4 = 0), operand 1 references the target address directly. When the operand is a PC-relative offset (CMB-4 = 1), operand 1 references the magnitude of the offset.

Operands 2 through 5 are not used by Call commands.

6.4 Notes

When a Call command executes, the processor saves protected call-frame state on the shadow call stack and then clears the active conditional-execution state for the called routine. This ensures that the called subroutine begins with a clean conditional-execution state regardless of how many conditional-execution levels were active at the call site. The saved state is restored when execution returns via the corresponding **Retrn** command.

This behavior differs from Jump commands, which use CMB 6–8 to encode a specific count of CEB levels to clear. A Jump may clear zero, some, or all active conditional-execution levels depending on the encoded count. A Call always saves and clears the current conditional-execution state as part of the broader protected call-frame save/restore behavior. CMB 6–8 are therefore unused in Call commands and must be 0.

This distinction reflects the different purposes of the two command families. Jump commands remain within the current execution context and may need to exit a specific number of conditional-execution levels. Call commands enter a new call context entirely, so protected call-frame save/restore plus a clean conditional-execution state is the appropriate behavior.

7. Data movement commands

7.1 Overview

Data movement commands transfer data between registers, memory, I/O ports, and the stack. Unlike arithmetic or logic commands, data movement commands do not transform data — they relocate it from a source to a destination, or between storage locations, without performing computational operations on the values being transferred.

This family encompasses several related sub-families, each addressing a different kind of data transfer:

- **Load and Store** — transfer data between memory and registers (section 7.2)

- **Push and Pull** — transfer data between registers and the stack (section 7.3)
- **Copy and Move** — transfer data between registers (section 7.4)
- **Read and Write** — transfer data between registers and hardware I/O ports (section 7.5)
- **Register utilities** — clear registers and swap register contents (section 7.6)

Each sub-family defines its own Command Modifier Bit usage and operand patterns. Some structural patterns recur across sub-families, but the specific CMB layout must be consulted for each sub-family individually.

7.2 Load and Store commands

Load and Store commands transfer data between memory and registers. Load commands read data from a memory address into a single destination register. Store commands write data from a source register to a single memory address. For the general instruction layout, refer to section 3.1.

The Load family provides direction-explicit assembler forms that differ both by alignment direction and by how the effective address is computed:

- **LoadL / LoadR** — effective address is the base address directly
- **LoadLOfs / LoadROfs** — effective address is $\text{baseAddr} + \text{offset}$
- **LoadLOfsMul / LoadROfsMul** — effective address is $\text{baseAddr} + (\text{offset} \times \text{multiplier})$

The Store family provides corresponding direction-explicit forms:

- **StorL / StorR** — effective address is the base address directly
- **StorLOfs / StorROfs** — effective address is $\text{baseAddr} + \text{offset}$
- **StorLOfsMul / StorROfsMul** — effective address is $\text{baseAddr} + (\text{offset} \times \text{multiplier})$

Direction is reflected in the assembler mnemonic and encoded through CMB-4 at the ISA level.

7.2.1 Command Modifier Bit usage

The Command Modifier Bits for Load and Store commands are organized as follows:

- **CMB 1-2:** Unused. These bits must be 0.
- **CMB-3:** Unused in current Load/Stor encoding.
- **CMB-4:** Alignment direction. Distinguishes the ***L** and ***R** assembler forms.
- **CMB-5:** Offset enable.
- **CMB-6:** Multiplier enable.

- **CMB 7-8:** Unused. These bits must be 0.

7.2.2 Operand usage

Operand 1 is always a byte count (**iR**) specifying how many bytes to transfer. This operand may be an immediate value or a register, as determined by CMB-3.

Load forms:

- **LoadL iR1 asR2 dR5 / LoadR iR1 asR2 dR5** — byte count, base address, destination register
- **LoadLOfs iR1 asR2 sR3 dR5 / LoadROfs iR1 asR2 sR3 dR5** — byte count, base address, offset, destination register
- **LoadLOfsMul iR1 asR2 sR3 sR4 dR5 / LoadROfsMul iR1 asR2 sR3 sR4 dR5** — byte count, base address, offset, multiplier, destination register

Store forms:

- **StorL iR1 sR2 adR3 / StorR iR1 sR2 adR3** — byte count, source register, base address
- **StorLOfs iR1 sR2 adR3 sR4 / StorROfs iR1 sR2 adR3 sR4** — byte count, source register, base address, offset
- **StorLOfsMul iR1 sR2 adR3 sR4 sR5 / StorROfsMul iR1 sR2 adR3 sR4 sR5** — byte count, source register, base address, offset, multiplier

Load commands always write to a single destination register (operand 5). Store commands always write to a single memory address. Neither Load nor Store supports fanout.

7.2.3 Notes

The offset and multiplier operands in the ***Ofs** and ***OfsMul** forms are currently register-only. They do not support immediate values.

In the current ISA model, offset and multiplier behavior is encoded through the CMB bits of the single **Load** or **Stor** ISA command, while the assembler exposes readable direction-explicit source forms such as **LoadLOfs**, **LoadROfs**, **StorLOfs**, and **StorROfs**.

7.3 Push and Pull commands

Push and Pull commands transfer data between registers and the stack. Kiss uses the terms **Push** and **Pull** rather than the more conventional Push and Pop. For the general instruction layout, refer to section 3.1.

A task, program, or thread in Kiss may have more than one stack. Operand 1 identifies which stack is used for the operation, and may be an immediate stack ID or a register-based selector.

Stack entries in Kiss are variable-sized. When data is pushed, a size field is appended to the data

on the stack (written after the data). When data is pulled, the processor reads the size field first — it lies at the top of the stack as the most recently written byte — and then unloads the corresponding number of bytes below it. This allows Push and Pull to handle varying data sizes without requiring the programmer to track sizes explicitly at the pull site.

7.3.1 Command Modifier Bit usage

The Command Modifier Bits for Push and Pull commands are organized as follows:

- **CMB 1-2:** Unused. These bits must be 0.
- **CMB-3:** Immediate-value indicator. When 0, operand 1 contains a register ID. When 1, operand 1 contains an immediate value (a stack ID).
- **CMB-4:** Unused. This bit must be 0.
- **CMB 5-6:** Unused. These bits must be 0.
- **CMB 7-8:** Immediate-value data type. When CMB-3 is 1, these bits encode the datatype of the immediate value in operand 1, following the convention described in section 3.4. When CMB-3 is 0, these bits are not meaningful.

7.3.2 Operand usage

- **Operand 1 (iR1):** Stack selector. Identifies which stack to use. This is an **iR** operand — it may be an immediate stack ID or a register containing the selector, as determined by CMB-3.
- **Operand 2 (R2):** First source register (Push) or first destination register (Pull). See operand patterns below for **sR/dR** distinction.
- **Operands 3-5 (oR3, oR4, oR5):** Optional additional registers. When present, these registers are processed consecutively after operand 2.

A single Push or Pull instruction can transfer up to four registers in one operation.

Operand patterns:

- **Push iR1 sR2 [osR3] [osR4] [osR5]** — stack selector, source register(s). Registers are pushed in operand order (R2 first, then R3, R4, R5).
- **Pull iR1 dR2 [odR3] [odR4] [odR5]** — stack selector, destination register(s). Registers are pulled in operand order (R2 first, then R3, R4, R5).

Note: Push and Pull do not follow the destination-last convention. The operand order matches the stack operation order — registers are processed sequentially from operand 2 through operand 5. This is necessary because the order of push/pull operations is semantically meaningful (stack ordering must be preserved).

7.3.3 Notes

The variable-sized stack entry format means that Pull does not require the caller to specify a byte count. The size information stored with each entry is sufficient for the processor to determine how much data to unload. This differs from Load commands, which require an explicit byte count in operand 1.

7.4 Copy and Move commands

Copy and Move commands transfer data between registers. Copy commands duplicate a source register value into one or more destination registers. Move commands do the same but additionally zero the source register after the transfer is complete. For the general instruction layout, refer to section 3.1.

The Copy family provides datatype- and direction-explicit assembler forms:

- **B.CopyL, I.CopyL, D.CopyL**
- **B.CopyR, I.CopyR, D.CopyR**

The Move family provides corresponding forms:

- **B.MoveL, I.MoveL, D.MoveL**
- **B.MoveR, I.MoveR, D.MoveR**

The datatype prefix determines how resizing and sign/decimal handling are interpreted when values move across different register widths.

7.4.1 Command Modifier Bit usage

The Command Modifier Bits for Copy and Move commands are organized as follows:

- **CMB 1-2:** Data type. These bits encode the datatype of the operation.
- **CMB-3:** Unused. This bit must be 0.
- **CMB-4:** Alignment direction. Distinguishes the ***L** and ***R** assembler forms.
- **CMB 5-8:** Unused. These bits must be 0.

7.4.2 Operand usage

Copy and Move commands do not use a leading byte-count operand. The source register is operand 1, and the remaining operands are destination registers. Optional trailing destinations allow a single instruction to fan out the source value to multiple registers.

Operand pattern:

- **Copy sR1 [odR2] [odR3] [odR4] dR5** — source, optional fanout destinations, primary

destination

The same operand pattern applies to all Copy and Move variants exposed by the assembler: `B/I/D.CopyL`, `B/I/D.CopyR`, `B/I/D.MoveL`, and `B/I/D.MoveR`.

7.4.3 Notes

For Move variants, the source register is zeroed only after all destination registers have been written. This ensures that the source value is fully available throughout the fan-out before it is cleared.

7.5 Read and Write commands (I/O)

Read and Write commands transfer data between registers and hardware I/O ports. These commands provide the mechanism for communicating with peripheral devices and other hardware-accessible ports. For the general instruction layout, refer to section 3.1.

Read and Write are type-agnostic — they move bytes without interpreting them, consistent with Load and Stor. The data type is determined by the instruction that later operates on the value.

The Read and Write family provides direction-explicit assembler forms:

- `ReadL` / `WriteL` — left-aligned transfer direction
- `ReadR` / `WriteR` — right-aligned transfer direction

The alignment direction is encoded via CMB-4.

7.5.1 Command Modifier Bit usage

The Command Modifier Bits for Read and Write commands are organized as follows:

- **CMB 1-2:** Unused. Must be `00`. Read/Write are type-agnostic.
- **CMB-3:** Operand 1 selector. When 0, byte count is in a register. When 1, byte count is an immediate value.
- **CMB-4:** Alignment direction. When 1, left-to-right (low-aligned). When 0, right-to-left (high-aligned). The `R` suffix variants correspond to CMB-4 = 0.
- **CMB 5-6:** Reserved (`00`).
- **CMB 7-8:** Unused. Must be `0`.

7.5.2 Operand usage

Both Read and Write use the same first two operands:

- **Operand 1 (`iR1`):** Byte count. Specifies the number of bytes to transfer. This is an `iR` operand — it may be an immediate value or a register, as determined by CMB-3.

- **Operand 2 (sR2):** Port selector. Identifies which I/O port to read from or write to.

The remaining operands differ by command:

- **Read:** Operand 5 (dR5) is the destination register that receives data from the port. Operands 3 and 4 are unused.
- **Write:** Operand 3 (sR3) is the source register whose contents are sent to the port. Operands 4 and 5 are unused.

Operand patterns:

- **Read iR1 sR2 dR5** — byte count, port selector, destination register
- **Write iR1 sR2 sR3** — byte count, port selector, source register

7.5.3 Notes

CMB-5/6 are reserved for future use (e.g., offset/multiplier addressing if register bank access patterns are needed).

7.6 Register utility commands

Register utility commands perform simple register-level operations that do not involve memory, the stack, or I/O ports. For the general instruction layout, refer to section 3.1.

This sub-family currently includes:

- **ClrReg** — clear one or more registers to zero
- **Swap** — exchange the contents of two registers

7.6.1 Command Modifier Bit usage

For both **ClrReg** and **Swap**, all Command Modifier Bits are unused and must be 0.

7.6.2 Operand usage

ClrReg:

- **Operand 5 (dR5):** Required. Primary register to clear.
- **Operands 4, 3, 2, 1 (odR4, odR3, odR2, odR1):** Optional additional registers to clear, filled backward from operand 4.

Each specified register is set to zero. A single **ClrReg** instruction can clear up to five registers. The destination-last convention applies: operand 5 is the required target, optional targets fill backward.

Swap:

- **Operand 1 (sdR1):** First register (read and written).
- **Operand 5 (sdR5):** Second register (read and written).

The values of the two registers are exchanged. Operands 2, 3, and 4 are not used.

7.7 Notes

The data movement sub-families share some structural patterns in their Command Modifier Bit usage. CMB-4 frequently serves as an alignment or direction selector, but the exact role of the remaining bits depends on the specific sub-family. Each sub-family therefore defines its own CMB layout, and the specific meaning of each bit must be determined from the relevant sub-family section.

SetVal and **SetTxt** are assembler mnemonics for data-loading commands that place immediate values into registers. Both compile to the single ISA command **SetImm** (CMB-4: **0**=SetVal/numeric, **1**=SetTxt/text). They use the current **SetImm** payload layout, with the destination in Operand 1 and the 50-bit payload carried in Operands 2–5, and are documented separately in section 13.

8. Arithmetic commands

8.1 Overview

The arithmetic command families perform computational operations on register values. All arithmetic commands are typed — each is prefixed with a datatype indicator (Binary, Integer, or Decimal) using dotted notation — and all share a common CMB structure referred to as Pattern A. See section 3.1 for the general instruction layout.

This section covers the following sub-families:

- **Ordinary arithmetic:** Add, Sub, Mul, Div
- **Fused multiply-add:** MulAdd
- **Divide with remainder:** DivRem
- **Apply-to families:** AddTo, SubTo, MulTo, DivTo
- **Increment and Decrement:** Incr, Decr

At the ISA level, these commands use untyped base names — **Add**, **Sub**, **Mul**, and so on — with the datatype encoded in the CMB bits. At the assembler level, dotted datatype-prefix notation is used — **B.Add** (Binary Add), **I.Sub** (Integer Subtract), **D.Mul** (Decimal Multiply) — which the assembler lowers into the corresponding ISA command with the appropriate CMB encoding.

8.2 Ordinary arithmetic (Add, Sub, Mul, Div)

8.2.1 Command Modifier Bit usage

The ordinary arithmetic commands share the following CMB assignment:

- **CMB 1-2:** Data type. **01** = Binary, **10** = Integer, **11** = Decimal.
- **CMB-3:** Immediate-value indicator. **0** = all operands are registers; **1** = immediate value present (see section 3.4).
- **CMB-4:** Saturation indicator. **0** = normal overflow/underflow behavior; **1** = saturated (clamped).
- **CMB-5:** Carry/borrow indicator. Command-specific; see below.
- **CMB-6:** Set Status Flags. **0** = do not update flags; **1** = update flags.
- **CMB 7-8:** Reserved. Must be **0** for ordinary arithmetic (used by the *To forms; see section 8.5).

CMB-5 usage by command:

- **Add:** When set to **1**, the current carry value is included as an additional arithmetic input (**AddCry**).
- **Sub:** When set to **1**, the current borrow value is included as an additional arithmetic input (**SubBrw**).
- **Mul, Div:** Not used. Must be **0**.

8.2.2 Assembler forms

At the ISA level, the base commands are simply **Add**, **Sub**, **Mul**, and **Div**, with the datatype and modifiers encoded in the CMB bits. The assembler provides typed mnemonic forms following the pattern **<Type>.<Op>[Cry][Brw][Sat][Set]**:

- Base forms: **B.Add**, **I.Add**, **D.Add**, **B.Sub**, **I.Sub**, **D.Sub**, **B.Mul**, **I.Mul**, **D.Mul**, **B.Div**, **I.Div**, **D.Div**
- Carry-aware (Add only): **B.AddCry**, **I.AddCry**, **D.AddCry**
- Borrow-aware (Sub only): **B.SubBrw**, **I.SubBrw**, **D.SubBrw**
- Saturating: append **Sat** — e.g., **B.AddSat**, **I.MulSat**
- Flag-setting: append **Set** — e.g., **D.DivSet**, **I.SubSet**

Suffixes combine in order: **B.AddCrySatSet**, **I.SubBrwSet**, **D.MulSatSet**.

8.2.3 Operand usage

- Minimum: 3 operands — two source registers and one destination register.
- Maximum: 5 operands — up to four source registers and one destination register.
- Optional middle operands provide additional input values; all are combined by the operation.
- The final operand is always the explicit destination register (**dR5**), which receives the result.

Operand pattern:

- **<Type>.Add sR1 sR2 [osR3] [osR4] dR5** — sources, destination

R5 = R1 + R2 (+ R3) (+ R4) — optional operands R3 and R4 are additional sources, not fanout destinations.

This operand pattern is identical for Add, Sub, Mul, and Div.

8.2.5 Carry / borrow chain math (CMB-5)

Setting CMB-5 = 1 enables carry-in / borrow-in chain semantics for multi-precision arithmetic. The chain register (**SCRY** for Add/Mul, **SBRW** for Sub/Div) is read as input and written with the new chain digit on output.

Type coverage and semantics by command:

- **Add (Bin / Int / Dec):** reads chain bit/digit from **SCRY**, computes **R1 + R2 + carry_in → R5**. Writes the “didn’t fit” / overflow digit back to **SCRY**. For Bin/Int the chain is single-bit; for Dec it’s a single 5-bit digit.
- **Sub (Bin / Int / Dec):** symmetric using **SBRW**. Computes **R1 – R2 – borrow_in → R5**.
- **Mul (Decimal only):** reads carry-in digit from **SCRY**, computes **(R1 × R2) + carry_in_at_LSD → R5**. Writes the highest digit lost beyond destination capacity back to **SCRY**. Intended for column-by-column long multiplication. Bin / Int Mul reject CMB-5 — multi-precision binary multiplication uses shift-and-add, not a single-bit chain.
- **Div (Decimal only):** reads borrow-in digit from **SBRW**. Computes the long-division step **(borrow_in × 10^DigitsR1 + R1) ÷ R2 → R5**, where **borrow_in** is **prepended at the HIGH end** of the dividend (standard long-division algorithm — different from Add/Sub/Mul which inject at the LSD). Writes the new remainder’s lowest digit back to **SBRW**. Bin / Int Div reject CMB-5 — multi-precision binary division uses bit-level long division.

MulAdd and **DivRem** reject CMB-5 (must be 0):

- **MulAdd**’s fused two-stage operation has no clean single-Cry chain semantic — a chain bit cannot unambiguously feed or capture both stages.

- **DivRem** already exposes the remainder via Operand 4; a separate Brw chain register would be redundant.

For Decimal Mul/Div with multi-digit second operand or divisor, the math is computed correctly but the single-digit chain register captures only one digit of the overflow/remainder — these operations are designed for the single-digit-column case typical of Decimal long arithmetic. (Per [docs/historical/decimal-update-plan.md](#) §Item 3, 2026-05-08 — supersedes the 2026-03-28 blanket “Cry/Brw not valid on Mul/Div” decision for Decimal only.)

8.3 Fused multiply-add (MulAdd)

8.3.1 Command Modifier Bit usage

MulAdd uses the same CMB layout as ordinary arithmetic (section 8.2.1). CMB-5 (carry/borrow) is not used and must be 0.

8.3.2 Behavior

MulAdd performs a fused multiply-add operation: the product of two operands is summed with a third, and the result is written to a destination register.

$(\text{multiplicand} \times \text{multiplier}) + \text{addend} \rightarrow \text{destination}$

Saturation (CMB-4) applies to the final result when enabled. CMB-6 controls whether status flags are updated.

8.3.3 Assembler forms

B.MulAdd, I.MulAdd, D.MulAdd, with optional **Sat** and **Set** suffixes — e.g., I.MulAddSat, D.MulAddSet, B.MulAddSatSet.

8.3.4 Operand usage

Operand pattern:

- **<Type>.MulAdd sR1 sR2 sR3 dR5** — multiplicand, multiplier, addend, destination

This is a fixed 4-operand form; no optional operands. Operand 4 is unused.

8.4 Divide with remainder (DivRem)

8.4.1 Command Modifier Bit usage

DivRem uses a CMB layout closely related to ordinary arithmetic, with one key difference at CMB-5:

- **CMB 1-2:** Data type. **01** = Binary, **10** = Integer, **11** = Decimal.
- **CMB-3:** Immediate-value indicator. Per section 3.4.

- **CMB-4:** Saturation indicator. 0 = normal; 1 = saturated.
- **CMB-5:** Divide-with-Remainder indicator. Always 1 — this distinguishes DivRem from plain Div.
- **CMB-6:** Set Status Flags. 0 = no; 1 = yes.
- **CMB 7-8:** Not otherwise defined. Subject to the global immediate-datatype rule when applicable.

8.4.2 Behavior

DivRem performs integer or typed division and produces both a quotient and a remainder, each written to a separate destination register.

8.4.3 Assembler forms

B.DivRem, I.DivRem, D.DivRem, with optional Sat and Set suffixes — e.g., I.DivRemSet, B.DivRemSatSet.

8.4.4 Operand usage

Operand pattern:

- **<Type>.DivRem sR1 sR2 dR4 dR5** — dividend, divisor, remainder destination, quotient destination

This is a fixed 4-operand form with two destination registers. There is no optional third divisor operand — unlike ordinary Div, DivRem always takes exactly two source operands.

8.4.5 Divide-by-zero behavior

The Bin/Int and Dec DivRem variants handle a zero divisor differently, matching the type’s representational capacity for infinity:

- **B.DivRem / I.DivRem with divisor = 0:** Faults (ARITH_OVERFLOW / divide-by-zero) unless CMB-4 (Sat) is set. With Sat, the quotient saturates by direction (max-positive on positive-direction overflow, 0 on negative-direction underflow) and the remainder receives 0. Binary and Integer have no representable infinity, so a “remainder of infinity” cannot be produced for these types.
- **D.DivRem with divisor = 0:** Never faults. Both quotient and remainder receive $\pm\infty$, with sign matching **dividend_sign XOR divisor_sign** — Kiss Decimal has signed zeros (+0, -0), so the divisor’s sign matters. The INF status flag is set when CMB-6 is 1. With Sat, both destinations instead clamp to all-9s of matching sign.

The “remainder = ∞ when divisor = 0” semantic for Decimal matches the HLL specification (HLL §4.1, **Dec Mod 0 = ∞**). Although a “remainder of infinity” is mathematically unusual (remainders are normally bounded by the divisor’s magnitude), infinity is the most useful runtime behavior in

practice: it propagates through subsequent arithmetic, amplifying rather than masking the error signal, and a divisibility check like $x \bmod n == 0$ correctly returns false when $n = 0$ (since infinity $\neq 0$). Returning 0 would falsely indicate “divisible”; trapping would kill the process for a recoverable runtime condition. Infinity is loud, propagating, and survivable — the right default for Decimal arithmetic where infinity is already a first-class value.

8.5 Apply-to families (AddTo, SubTo, MulTo, DivTo)

8.5.1 Kiss-50 Command Modifier Bit usage

- **CMB 1-2:** datatype.
- **CMB-3:** unused; must be 0 (Set not valid).
- **CMB-4:** Saturation.
- **CMB-5:** operand-1 value selector: 0 = value register, 1 = compact immediate value.

8.5.2 Assembler forms

B.AddTo, I.AddTo, D.AddTo, B.SubTo, I.SubTo, D.SubTo, B.MulTo, I.MulTo, D.MulTo, B.DivTo, I.DivTo, D.DivTo, with optional Sat suffix only. Set, Cry, and Brw are not valid for Apply-to.

8.5.3 Operand usage

Operand pattern:

- $\langle \text{Type} \rangle.\text{AddTo } iR1 \text{ } sR2$ — value to apply and in-place target. CMB-5 selects compact immediate value versus value register. The target is read, the operation applied, and the result written back in place.

The same pattern applies to SubTo, MulTo, and DivTo.

8.6 Increment and Decrement (Incr, Decr)

8.6.1 Command Modifier Bit usage

In compact Kiss-50, Incr and Decr use the datatype and SetFlags bits plus a family-specific operand-1 immediate selector:

- **CMB 1-2** — datatype.
- **CMB-3** — Set Status Flags.
- **CMB-4** — not used; must be 0.
- **CMB-5** — operand-1 amount selector: 0 = amount register, 1 = compact immediate amount.

Sat, Cry, and Brw suffixes remain invalid for Incr/Decr.

8.6.2 Behavior

Incr and Decr are amount-driven update forms. Operand 1 specifies the amount. Operand 2 is the Kiss-50 target register, read/updated/written in place.

8.6.3 Assembler forms

B.Incr, I.Incr, D.Incr, B.Decr, I.Decr, D.Decr, with optional Set suffix — e.g., I.IncrSet, D.DecrSet.

Because CMB-4 is unused and CMB-5 is the amount immediate selector, Sat, Cry, and Brw suffixes do not apply to Incr or Decr.

8.6.4 Operand usage

Operand pattern:

- <Type>.Incr iR1 sdR2 — amount in operand 1 and target in operand 2. CMB-5 selects compact immediate amount versus amount register. The target is read, updated by the amount, and written back in place.

The same pattern applies to Decr, subtracting the amount.

8.7 Notes

All arithmetic families are typed. CMB 1–2 always encodes the datatype (Binary, Integer, or Decimal), and the assembler mnemonic always carries the corresponding type prefix.

The arithmetic families share Pattern A as their common CMB structure: CMB 1–2 for datatype, CMB-6 for Set Status Flags (where supported), with the remaining bits carrying family-specific meanings as documented in each sub-section above.

The A.* family commands that may appear in assembler command listings (e.g., A.Add, A.Sub) are simulator and VM helper commands. They are **not** part of the Kiss ISA and must not be confused with the typed arithmetic commands described in this section.

Saturation means that on overflow, the result is clamped to the maximum representable value for the active datatype and register width; on underflow, the result is clamped to the minimum representable value. Saturation never wraps.

9. Rotate and shift commands

9.1 Overview

The Kiss ISA provides three command families for bitwise rotation and shifting:

ISA Command	Description
Rot	Rotate
Shift	Logical shift
SShift	Signed (arithmetic) shift

Rot and **Shift** are typed commands accepting all three data types. **SShift** accepts Integer only — Binary has no sign concept, and Decimal’s signed-digit encoding (sign embedded in every digit) makes a “preserve the sign bit at MSB” semantic ill-defined. All three commands share an identical CMB layout and accept direction and status-flag modifiers. For the general instruction layout, refer to section 3.1.

At the ISA level, these commands use untyped base names (**Rot**, **Shift**, **SShift**). All behavioral variation — data type, direction, and flag behavior — is encoded in the CMB bits. The assembler provides expanded, human-readable command names (e.g., **B.RotL**, **I.ShiftRSet**) that it lowers into the corresponding ISA command with the appropriate CMB encoding.

9.2 Command Modifier Bit usage

The following CMB layout is shared across all three command families:

- **CMB 1-2:** Data type. **01** = Binary, **10** = Integer, **11** = Decimal.
- **CMB-3:** Immediate-value indicator (operand 1). **0** = register, **1** = immediate.
- **CMB-4:** Direction. **0** = Right, **1** = Left.
- **CMB-5:** Unused. Must be **0**. Through-carry rotate/shift was retired across the family — RISC-V and ARM AArch64 both omit through-carry rotate; Kiss follows that convention. Multi-precision arithmetic uses **Add+Cry** / **Sub+Brw** chain math instead.
- **CMB-6:** Set Status Flags. **0** = no, **1** = yes.
- **CMB 7-8:** Immediate-value data type. Standard encoding per section 3.4 (applies when CMB-3 = **1**). When CMB-3 = **0**, these bits are unused.

9.3 Assembler command names

9.3.1 Naming pattern

The assembler uses the following naming convention:

<Type>.<Family><Direction>[Set]

Where:

- **Type** — data-type prefix: **B** (Binary), **I** (Integer), or **D** (Decimal). **SShift** accepts only **I**.
- **Family** — **Rot**, **Shift**, or **SShift**

- **Direction** — **L** (Left) or **R** (Right)
- **Set** — optional; indicates that status flags are set on completion

9.3.2 Naming notes

- **Set** is used as the readable suffix for set-status-flags.
- The leading **S** in **SShift** is part of the command-family name (Signed Shift). It does **not** indicate Set Status Flags. A trailing **Set** is the only indicator for that modifier.

9.3.3 Examples

Assembler Name	Expansion	ISA Command	Key CMB Settings
B.RotL	Binary Rotate Left	Rot	type=Binary, dir=Left
I.ShiftR	Integer Shift Right	Shift	type=Integer, dir=Right
D.RotL	Decimal Rotate Left (digit-level)	Rot	type=Decimal, dir=Left
D.ShiftRSet	Decimal Shift Right (digit-level), Set Flags	Shift	type=Decimal, dir=Right, set=yes
I.SShiftL	Integer Signed Shift Left	SShift	type=Integer, dir=Left
I.SShiftRSet	Integer Signed Shift Right, Set Flags	SShift	type=Integer, dir=Right, set=yes

Each assembler form is lowered into one of the three ISA-level commands (**Rot**, **Shift**, or **SShift**) with the corresponding CMB bits set to encode the type, direction, and flag behavior.

9.3.4 Current assembler coverage

Rot and **Shift** are defined for all three data types in both directions. **SShift** is defined for Integer in both directions. The full mnemonic set is $\{B, I, D\} \times \{Rot, Shift\} \times \{L, R\}$ plus $I \times SShift \times \{L, R\}$, with the optional **Set** suffix on every form.

9.4 Operand usage

All three command families share the same operand layout, following the destination-last convention for in-place operations:

- **Operand 1 (iR1):** Required. Shift/rotate count — immediate or register per CMB-3.
- **Operand 5 (sdR5):** Required. Primary target register to be rotated/shifted (in-place: both source and destination).
- **Operands 4, 3, 2 (osdR4, osdR3, osdR2):** Optional additional target registers, filled backward from operand 4.

The count specified by operand 1 is applied uniformly to every target register present in the instruction. A single instruction can therefore rotate or shift up to four registers by the same count. Each register is independently modified in-place.

9.5 Behavioral notes

- **Rot** performs a circular rotation.
 - **Binary / Integer:** bit-level rotation. Bits shifted out of one end re-enter at the opposite end.
 - **Decimal:** digit-level rotation. 5-bit packed digits shifted out of one end re-enter at the opposite end. **Null**-coded digits are skipped — they remain in their physical positions while the non-Null digits rotate around them.
- **Shift** performs a logical shift. Bits/digits shifted off the end are lost.
 - **Binary / Integer:** bit-level shift. Vacated positions filled with **0**.
 - **Decimal:** digit-level shift. Vacated digit positions filled with the sign-matched zero digit (**Pos_0** if the value's sign — taken from the first non-Null digit — is positive, **Neg_0** if negative). **Null**-coded digits and the decimal-point marker are skipped during the shift. Lost non-Null digits are silent — the Shift / Rot family does not set **Ovr** / **Und** / **Wrp** on bit-loss (decimal-update-plan Spec C, 2026-05-08).
- **SShift** (Signed Shift) performs an arithmetic shift that preserves the sign of the value. The sign bit (high-order bit, bit 1 per Kiss convention) is never modified by the shift — it remains in place regardless of direction. Only the magnitude bits move. **Vacated magnitude positions fill with the inverse of the sign bit** — **0** for positive Int (sign bit **1**), **1** for negative Int (sign bit **0**). This rule applies to both directions: **SShiftR** vacates high-order magnitude positions, **SShiftL** vacates low-order magnitude positions, and both fill with **NOT(sign bit)**. The rule is the inverse of x86's "fill with sign bit" SAR semantic — Kiss reverses it because Kiss's sign convention is reversed (sign-bit-1 = positive). Lost magnitude bits are silent — the Shift / Rot family does not set **Ovr** / **Und** / **Wrp** on bit-loss (decimal-update-plan Spec C, 2026-05-08). This makes **SShift** appropriate for signed multiplication or division by powers of two and other sign-preserving operations.
- When CMB-6 (Set Status Flags) is enabled, the status flags are updated to reflect the result of the operation after all specified target registers have been processed.
- All three families respect the data-type width indicated by CMB 1-2. The rotation or shift operates on the full width of the specified type.

10. Logic and bit-processing commands

10.1 Overview

This section covers two related groups of commands:

1. **Logic families** — And, AndN, NAnd, Or, OrN, NOOr, XOr, NXOr, Not, Neg

2. Bit-processing commands — BitFlip, BitOn, BitOff, BitTstOn, BitTstOff

All logic families are typed commands that share the same CMB layout — the simplest typed layout in the ISA, encoding only the data type and an optional Set Status Flags modifier. Bit-processing commands are conceptually related; some are fully specified, others remain provisional.

As with all typed commands in Kiss, the ISA-level command uses an untyped base name (e.g., **And**, **Not**, **Neg**), and the data type is encoded in the CMB bits of the instruction word (see section 3.1). The dotted forms seen in assembly source — **B.And**, **I.Not**, **D.NegSet**, and so on — are assembler command names. The assembler lowers each dotted form into the corresponding ISA command plus the appropriate CMB encoding.

10.2 Logic families — CMB layout

All logic families, including Not and Neg, share a single CMB layout:

- **CMB 1-2:** Data type. **01** = Binary, **10** = Integer, **11** = Decimal.
- **CMB 3-5:** Unused. Must be **0**.
- **CMB-6:** Set Status Flags. **0** = no, **1** = yes.
- **CMB 7-8:** Unused. Must be **0**.

This is the simplest typed CMB layout in the ISA: data type selection and an optional status-flag update. No other modifiers apply. See section 3.4 for data type and immediate-value conventions.

When CMB-6 is set to **1**, the assembler form appends **Set** to the command name (e.g., **I.AndSet**, **D.NotSet**). Without the **Set** suffix, status flags are left unchanged by the operation.

10.3 Ordinary logic — And, AndN, NAnd, Or, OrN, NOr, XOr, NXOr

10.3.1 Description

These eight commands perform standard two-input logic operations. Each takes two source registers and writes the result to a destination register.

ISA Command	Operation
And	Bitwise AND of two operands
AndN	Bitwise AND-NOT (AND of operand 1 with the complement of operand 2)
NAnd	Bitwise NAND (complement of AND)
Or	Bitwise OR of two operands
OrN	Bitwise OR-NOT (OR of operand 1 with the complement of operand 2)
NOr	Bitwise NOR (complement of OR)
XOr	Bitwise exclusive OR

ISA Command	Operation
NXOr	Bitwise exclusive NOR (complement of XOR)

All eight commands use the same operand pattern and the shared CMB layout described in section 10.2. The operation is applied at the width determined by the encoded data type.

10.3.2 Operand usage

- **Operand 1 (sR1):** First input register.
- **Operand 2 (sR2):** Second input register.
- **Operand 5 (dR5):** Destination register (receives the result).

Operands 4 and 5 are not used.

10.3.3 Assembler forms

The assembler command name combines a data-type prefix with the base command, and optionally the **Set** suffix:

- **Type prefix:** **B.** (Binary), **I.** (Integer), **D.** (Decimal)
- **Set suffix:** appended when the instruction should update status flags

Examples: **B.And**, **I.OrSet**, **D.XOr**, **B.NAndSet**.

The assembler encodes the type prefix into CMB 1–2 and the **Set** suffix into CMB-6. The resulting ISA instruction uses the untyped base command with the appropriate CMB value.

10.4 Not

10.4.1 Description

Not performs bitwise inversion (one's complement). Unlike the ordinary logic commands, Not operates **in-place** — each operand register is both the source and the destination of the inversion. When the Set modifier is not active, Not can invert up to five registers in a single instruction.

10.4.2 Operand usage

Without Set (CMB-6 = 0):

- **Operand 5 (sdR5):** Required register to invert (in-place: both source and destination).
- **Operands 4, 3, 2, 1 (osdR4, osdR3, osdR2, osdR1):** Optional additional registers to invert, filled backward from operand 4.

Each listed register is inverted in-place. Up to five registers may be specified.

With Set (CMB-6 = 1):

- **Operand 5 (sdR5):** Single register to invert (required).

When Set is active, only one register is permitted. Status flags must reflect a single deterministic result; allowing multiple registers would leave the flag state ambiguous.

10.4.3 Assembler forms

Examples: **B.Not**, **I.Not**, **D.NotSet**.

Because Not inverts in-place, the assembler form lists only the register(s) to be inverted — there is no separate destination operand.

10.5 Neg

10.5.1 Description

Neg performs arithmetic negation (sign inversion) **in place**: the target register is read, its negated value is computed, and the result is written back to the same register. Neg takes a single register operand and does not support fanout.

10.5.2 Data-type restriction

Arithmetic negation is meaningful only for signed types. The Binary data type is unsigned; consequently, **B.Neg** is not a valid assembler form. The valid assembler forms are:

- **I.Neg**, **I.NegSet** — Integer negation
- **D.Neg**, **D.NegSet** — Decimal negation

An encoding with CMB 1-2 = 01 (Binary) and the Neg opcode is reserved and must not be emitted.

10.5.3 Operand usage

- **Operand 5 (sdR5):** Target register (read, negated, and written back in place).

Operands 1 through 4 are unused. Neg does not support fanout; only a single target register is permitted.

10.5.4 Not vs. Neg

Both commands are single-register in-place operations but differ in semantics and fanout:

Aspect	Not	Neg
Operation	Bitwise inversion (one's complement)	Arithmetic negation (sign inversion)
Destination	In-place — target register is read, inverted, and written back	In-place — target register is read, negated, and written back

Aspect	Not	Neg
Multi-register	Up to 5 registers inverted independently (in-place fanout)	Single target only (no fanout)
Binary type	Valid (B.Not)	Invalid — unsigned values have no arithmetic negation

10.6 BitFlip

BitFlip is semantically identical to Not — it performs a bitwise inversion of the operand value. It exists as a distinct ISA command to provide an explicit bit-manipulation name alongside the logic-family Not, but the underlying operation (one's complement) is the same.

BitFlip uses the same CMB layout as the logic families (section 10.2). Its operand pattern and Set-modifier behavior follow the same rules as Not (section 10.4.2).

10.7 BitOn, BitOff, BitTstOn, BitTstOff

These four commands provide targeted single-bit manipulation and testing:

Command	Operation
BitOn	Set a specific bit to 1 (assembler for ISA BitSet with CMB-4=1)
BitOff	Clear a specific bit to 0 (assembler for ISA BitSet with CMB-4=0)
BitTstOn	Test whether a specific bit is 1 — sets Z flag (assembler for ISA BitTst with CMB-4=1)
BitTstOff	Test whether a specific bit is 0 — sets Z flag (assembler for ISA BitTst with CMB-4=0)

10.7.1 Bit-addressing model

The bit position is specified via **Option C**: CMB-3 selects whether operand 1 is an immediate value (**1**) or a register (**0**) containing the bit position. This is consistent with other command families that use CMB-3 as the immediate-value indicator.

Bit positions are **1-based** per the Kiss indexing directive. Bit 1 is the least significant bit. With 2000-bit register banks, positions range from 1 to 2000 (11 bits, fits in a 15-bit operand field).

10.7.2 Operand usage

BitOn, BitOff:

- **Operand 1 (iR1)**: Bit position — immediate or register (CMB-3 selects).
- **Operand 5 (sdR5)**: Target register (modified in place). Single target only — no fanout.

BitTstOn, BitTstOff:

- **Operand 1 (iR1)**: Bit position — immediate or register (CMB-3 selects).
- **Operand 5 (sR5)**: Source register (not modified).

Status flags: **BitTstOn** sets Z=1 if the tested bit IS 1. **BitTstOff** sets Z=1 if the tested bit IS 0. The

programmer uses **Jumplf Z** to branch on the test result.

10.8 Bit Field Operations — BitRead, BitWrite, BitClear

These three commands operate on arbitrary-width groups of bits within a register. They are the multi-bit generalizations of **BitOn/BitOff/BitTstOn/BitTstOff**.

Command	Operation
BitRead	Read (extract) a group of bits from a register — source untouched, result low-aligned in dest, upper bits zeroed
BitWrite	Write (insert) a group of bits into a register at a position — other bits in target unchanged, no shifting
BitClear	Clear (zero) a group of bits at a position — other bits unchanged

10.8.1 CMB layout

Shares the bit-processing CMB layout:

- **CMB 1-2:** **00** (untyped — bit operations have no data type selection).
- **CMB-3:** Immediate-value indicator for operand 1 (start position). **0** = register, **1** = immediate.
- **CMB-4:** Immediate-value indicator for operand 2 (width). **0** = register, **1** = immediate.
- **CMB-5:** Reserved.
- **CMB-6:** Set Status Flags. **0** = no, **1** = yes.
- **CMB 7-8:** Reserved.

10.8.2 Operand usage

BitRead:

- **Operand 1 (iR1):** Start position — immediate or register (CMB-3 selects).
- **Operand 2 (iR2):** Width — immediate or register (CMB-4 selects).
- **Operand 3 (sR3):** Source register (not modified).
- **Operand 4 (dR4):** Destination register (receives low-aligned result).

BitWrite:

- **Operand 1 (iR1):** Start position.
- **Operand 2 (iR2):** Width.
- **Operand 3 (sR3):** Source register (low-aligned value to write).
- **Operand 4 (sdR4):** Target register (modified in place).

BitClear:

- **Operand 1 (iR1):** Start position.
- **Operand 2 (iR2):** Width.
- **Operand 3 (sdR3):** Target register (modified in place).

10.8.3 Naming rationale

“Read” and “Write” accurately describe the semantics. “Extract” and “Insert” (from x86 **BEXTR** / ARM **BFI**) imply removal and insertion with shifting, which is not what these operations do. The x86/ARM “BF” (Bit Field) prefix is dropped as unnecessary jargon.

10.9 Bit Counting and Finding — **BitCntTotOn**, **BitCntTotOff**, **BitCntLeadOff**, **BitCntLeadOn**, **BitCntTrailOff**, **BitCntTrailOn**, **BitFindFstOn**, **BitFindLstOn**

These commands count or locate set/clear bits within a register.

Command	Operation
BitCntTotOn	Count total set bits — assembler for ISA BitCntTot (CMB-4=1)
BitCntTotOff	Count total clear bits — assembler for ISA BitCntTot (CMB-4=0)
BitCntLeadOff	Count leading zeros (from MSB) — assembler for ISA BitCnt (CMB-3=1, CMB-4=0)
BitCntLeadOn	Count leading ones — assembler for ISA BitCnt (CMB-3=1, CMB-4=1)
BitCntTrailOff	Count trailing zeros (from LSB) — assembler for ISA BitCnt (CMB-3=0, CMB-4=0)
BitCntTrailOn	Count trailing ones — assembler for ISA BitCnt (CMB-3=0, CMB-4=1)
BitFindFstOn	Find first set bit from left — assembler for ISA BitFind (CMB-3=1, CMB-4=1)
BitFindLstOn	Find last set bit from right — assembler for ISA BitFind (CMB-3=0, CMB-4=1)

10.9.1 Operand usage

All seven commands share the same operand layout:

- **Operand 1 (sR1):** Source register (not modified).
- **Operand 5 (dR5):** Destination register (receives the count or position).

10.9.2 “Not found” result for BitFind (**BitFindFstOn** / **BitFindLstOn**)

If no set bit is found, the result is 0 — an invalid position per Kiss’s 1-based indexing directive. This provides a natural “not found” sentinel. The assembler **BitFindFstOn** / **BitFindLstOn** forms encode CMB-6 Set, so the Z flag is set on not-found and **Jumplf Z** branches on “not found.”

10.10 Bit and Byte Reversal — **BitRev**, **ByteRev**

Command	Operation
BitRev	Reverse bit order within a register

Command	Operation
ByteRev	Reverse byte order within a register (endianness swap)

Note: **ByteRev** uses a **Byte** prefix, not **Bit**, because it operates on Kiss bytes (10-bit units), not individual bits.

10.10.1 Operand usage

- **Operand 1 (sR1):** Source register (not modified).
- **Operand 5 (dR5):** Destination register (receives reversed result).

10.11 Notes

Shared CMB simplicity. All logic families use the same two-field CMB layout (data type + Set Status Flags). No command in this section requires additional CMB modifiers, making the logic group the simplest typed command family in the ISA.

Set restriction on Not and Neg. When Set is active, both Not and Neg restrict their operand count to a single result value. This constraint ensures that the status flags reflect one deterministic outcome. Allowing multi-register operations to drive flag updates would produce ambiguous or implementation-dependent flag state.

BitFlip redundancy. BitFlip duplicates Not at the functional level. The separate command exists to provide a semantically explicit name within the bit-processing family, distinct from the logic-family Not. Implementations may share the same execution path for both.

Bit* prefix convention. All bit manipulation instructions use the **Bit*** prefix (no dot separator) to distinguish them from data-type-prefixed commands (**B.***, **I.***, **D.***). Since bit operations are inherently binary, the **Bit** prefix replaces the data-type prefix. The one exception is **ByteRev**, which uses a **Byte** prefix because it operates on Kiss bytes, not individual bits.

CMB 1-2 = 00 as type indicator. The bit-processing family uses **00** in the data type CMB bits, which is otherwise unused (**01**=Binary, **10**=Integer, **11**=Decimal). This provides a clean fourth encoding for the instruction decoder to route to bit-processing hardware.

11. Test, compare, and bounds commands

11.1 Overview

Three ISA commands are currently implemented in this family: **Test**, **Compare**, and **ChkBnds**. A fourth, **CmpMag** (magnitude compare), is spec-defined but not yet implemented; it is documented in §11.8 for reference and to reserve its place in the family. All four are typed commands that evaluate conditions and set status flags. They do not produce a result in a destination register — their sole purpose is flag-setting.

At the ISA level, the command names are **Test**, **Compare**, **ChkBnds**, and (future) **CmpMag**. The

datatype is encoded in CMB 1-2, and the assembler provides dotted datatype-prefix forms (e.g., **I.Test**) that lower into the base ISA command with the appropriate CMB encoding (see section 3.4).

The Compare-group flags split into a shared equality flag (**EQ**, bit 13), a directional quartet for natural-ordering compares (**GT/GE/LT/LE**, bits 14-17), and a directional quartet for magnitude compares (**Abv/AbvE/Blw/BlwE**, bits 18-21). **Compare** and **ChkBnds** write only **EQ** and the directional quartet; **CmpMag** writes only **EQ** and the magnitude quartet. The two directional quartets are disjoint, so a **Compare** followed by a **CmpMag** leaves both readable side by side.

Kiss's data type encodings are designed so that raw register-level ordering coincides with semantic ordering for every type (Bin / Int / Dec / Text). **Compare** therefore performs a single raw register comparison and produces correct ordering for any datatype — there is no x86-style “signed compare vs unsigned compare” distinction at the ISA level on Kiss. The CMB 1-2 datatype field on **Compare** is informational; the comparator behavior does not change with it. See the ISA reference, “Compare Flags” subsection, for the full design rationale.

11.2 Command Modifier Bit usage

These commands share the same CMB layout used by the logic families, with one difference: because Test, Compare, and ChkBnds exist solely to set status flags, CMB-6 is **required** to be **1** for every instance of this family.

- **CMB 1-2:** Data type. **01** = Binary, **10** = Integer, **11** = Decimal.
- **CMB 3-5:** Unused. Must be **0**.
- **CMB-6:** Set Status Flags. **Must be 1** for this family.
- **CMB 7-8:** Unused. Must be **0**.

11.3 Test

The **Test** command compares a single operand against zero and sets status flags according to the result.

- **Operand 1 (sR1):** Value to test against zero.

The processor evaluates sR1 relative to zero using the datatype specified by CMB 1-2 and sets the status flags to reflect the outcome (e.g., equal-to-zero, greater-than-zero, less-than-zero).

Only operand 1 is used; remaining operand fields are unused.

11.4 Compare

The **Compare** command compares two operands and sets status flags according to the relationship between them.

- **Operand 1 (sR1):** First comparand.
- **Operand 2 (sR2):** Second comparand.

The processor evaluates sR1 relative to sR2 — a raw register comparison, which is the natural-ordering compare for whatever datatype CMB 1-2 specifies (see §11.1) — and sets the **shared equality flag (EQ)** plus the **directional quartet (GT/GE/LT/LE)** to reflect the result. The magnitude quartet (**Abv/AbvE/Blw/BlwE**) is **not** touched by **Compare** — it is set only by **CmpMag** (§11.8).

11.5 ChkBnds (Check Bounds)

The **ChkBnds** command tests whether a value lies inclusively between a low bound and a high bound.

- **Operand 1 (sR1):** Value to check.
- **Operand 2 (sR2):** Low bound (inclusive).
- **Operand 3 (sR3):** High bound (inclusive).

The processor evaluates whether $sR1 \geq sR2$ and $sR1 \leq sR3$ using the datatype specified by CMB 1-2 (raw register comparison, per the universal-ordering property — see §11.1), and sets **EQ** plus the directional quartet (**GT/GE/LT/LE**) to reflect whether the value is within bounds, below the low bound, or above the high bound. As with **Compare**, the magnitude quartet is not touched.

11.6 Assembler forms

The assembler provides dotted datatype-prefix forms that lower into the base ISA command with the appropriate CMB 1-2 encoding:

Assembler Form	ISA Command
B.Test, I.Test, D.Test	Test
B.Compare, I.Compare, D.Compare	Compare
B.ChkBnds, I.ChkBnds, D.ChkBnds	ChkBnds
I.CmpMag, D.CmpMag (<i>future</i>)	CmpMag (<i>spec-defined but not yet implemented</i>)

11.7 Notes

- These commands do not write to a destination register. They only evaluate and set flags. Conditional Execution Bits on subsequent instructions can then branch on or conditionally execute based on those flags.
- The CMB layout is identical to the one used by the logic command families. This shared layout simplifies decoding and keeps CMB-6 as a uniform “set status flags” control across multiple command families.
- All operand registers are read-only (**sR** notation) — no operand is modified by these

commands.

11.8 CmpMag (Magnitude Compare) — spec-defined, not yet implemented

Status: spec-defined but not yet implemented. Flag bits 18–21 (*Abv/AbvE/Blw/BlwE*) are reserved for *CmpMag*; no current Kiss instruction writes them.

The *CmpMag* command compares the **magnitudes** of two operands ($|sR1|$ vs $|sR2|$, ignoring sign) and sets the magnitude directional quartet of the Compare-group flags accordingly. Distinct from *Compare*, which is a signed compare.

- **Operand 1 (sR1):** First comparand.
- **Operand 2 (sR2):** Second comparand.

The processor evaluates $|sR1|$ relative to $|sR2|$ using the datatype specified by CMB 1–2 and sets:

Flag	Set when
EQ	$ sR1 = sR2 $ (shared equality flag — overwrites any prior Compare/ChkBnds EQ value)
Abv	$ sR1 > sR2 $
AbvE	$ sR1 \geq sR2 $
Blw	$ sR1 < sR2 $
BlwE	$ sR1 \leq sR2 $

The signed directional quartet (*GT/GE/LT/LE*) is **not** touched by *CmpMag*. A *Compare* followed by a *CmpMag* (or vice versa) leaves both directional quartets independently readable; only *EQ* is shared and reflects whichever compare ran most recently.

Type coverage. Signed types only: *I.CmpMag* (Integer) and *D.CmpMag* (Decimal). For Binary (unsigned) values, $|x| \equiv x$, so a *B.CmpMag* would produce identical results to *B.Compare* — the encoding *CMB 1-2 = 01* (Binary) is reserved/illegal for *CmpMag*.

CMB layout. Identical to *Compare*:

- **CMB 1-2:** Data type. *10* = Integer, *11* = Decimal. (*01* = Binary is reserved.)
- **CMB 3-5:** Unused. Must be *0*.
- **CMB-6:** Set Status Flags. Must be *1*.
- **CMB 7-8:** Unused. Must be *0*.

Use cases (motivation).

- Iterative numerical solvers — convergence tests like $|residual| < tolerance$ without first computing absolute values into a temporary register.
- AI-core activation gating — $|x| > threshold$ for ReLU-family activations, attention masking, and quantization decisions.

- Magnitude-sort algorithms — ordering a sequence by $|x|$ rather than signed value.
- Error-bound monitoring — $|\text{measured} - \text{target}| \leq \epsilon$ checks in feedback loops.

Why a separate command rather than a Compare mode flag. Magnitude comparison requires absolute-value extraction on each operand before the subtraction; it is not a free re-reading of subtractor output. Folding it into Compare would force every compare to pay the absolute-value extraction cost (or branch on a CMB bit), and would tie EQ-vs-magnitude semantics to a CMB option, which compromises decoder regularity. A dedicated opcode keeps Compare simple and CmpMag's cost paid only when it is wanted.

Why no mainstream ISA has it. x86, ARM, MMIX, and RISC-V all provide signed and unsigned compare but not magnitude compare; magnitude is synthesized from `abs` + `cmp`. Kiss provides it as a primitive because the AI-core and numerical-algorithm workloads on the roadmap exercise it on hot paths, where a one-instruction primitive earns its keep.

12. Conversion commands

12.1 Overview

The `Cvt` (Convert) command converts a value from one data type to another. It is a single ISA command that handles all supported conversion paths by encoding both the source and destination types in the CMB field.

At the ISA level, the command name is simply `Cvt`. The assembler provides multi-type dotted notation (e.g., `B.I.Cvt`) that lowers into the base command with the appropriate CMB encoding.

12.2 Command Modifier Bit usage

The `Cvt` command uses a unique CMB layout in which both CMB 1-2 and CMB 7-8 encode data types, but with different roles:

- **CMB 1-2:** Source data type. `01` = Binary, `10` = Integer, `11` = Decimal.
- **CMB 3-6:** Unused. Must be `0`.
- **CMB 7-8:** Destination data type. `01` = Binary, `10` = Integer, `11` = Decimal.

This dual-type encoding is unique among Kiss commands. No other command uses CMB 1-2 and CMB 7-8 for two different datatype roles. The design collapses all conversion paths (Binary $\leftrightarrow$ Integer, Integer $\leftrightarrow$ Decimal, Binary $\leftrightarrow$ Decimal, etc.) into a single ISA command opcode.

12.3 Assembler forms

The assembler uses a multi-type dotted notation with the pattern:

`<SourceType>.<DestType>.Cvt`

The assembler encodes the source type into CMB 1-2 and the destination type into CMB 7-8. The ISA command emitted is always **Cvt**.

Examples:

Assembler Form	Source Type	Dest Type
B.I.Cvt	Binary	Integer
I.D.Cvt	Integer	Decimal
D.B.Cvt	Decimal	Binary
I.B.Cvt	Integer	Binary
B.D.Cvt	Binary	Decimal
D.I.Cvt	Decimal	Integer

Since Kiss Decimal IS packed decimal, there are no separate Packed Decimal conversion commands — **D.B.Cvt**, **D.I.Cvt**, etc. already handle conversions to/from the packed decimal format.

12.4 Operand usage

The **Cvt** command uses two operands:

- **Operand 1 (sR1):** Source register containing the value to convert (read, not modified).
- **Operand 5 (dR5):** Destination register to receive the converted value.

The processor reads the value in sR1, interprets it according to the source datatype (CMB 1-2), converts it to the destination datatype (CMB 7-8), and writes the result to dR5. The source register sR1 is not modified. Operands 2 through 4 are not used. No fanout — single source, single destination.

12.5 Notes

- **Cvt** is the only Kiss ISA command that uses CMB 1-2 and CMB 7-8 for two different datatype roles. All other typed commands use only CMB 1-2 for a single datatype.
- The same-type conversion case (e.g., CMB 1-2 = **01** and CMB 7-8 = **01**, Binary to Binary) is encodable but semantically a no-op copy. Whether the processor treats this as valid or raises a fault is an implementation decision.
- If Packed Decimal later requires encoding beyond the three standard values in CMB 1-2, the CMB layout can accommodate it without affecting the base **Cvt** command structure.

13. Special cases

13.1 SetVal and SetTxt

The **SetVal** and **SetTxt** assembler commands load an immediate value directly into a register.

Both compile to the single ISA command **SetImm**. **SetImm is a special-form command** — the destination is carried in Operand 1 (not Operand 5), and the 50-bit payload occupies Operands 2–5 (right-justified; bits 41–50 are unused padding).

13.1.1 Command Modifier Bit usage

- **CMB 1-2:** Data type. **00** = untyped, **01** = Binary, **10** = Integer, **11** = Decimal. Used by the CPU to apply type-aware width extension when the destination register is wider than 50 bits — see §13.1.3 below.
- **CMB-3:** Immediate-form indicator. Always **1** for **SetImm**.
- **CMB-4:** Numeric/text selector. **0** = SetVal, **1** = SetTxt.
- **CMB 5-8:** Unused. Must be **0**.

CMB-4 distinguishes the two immediate payload interpretations:

- **CMB-4 = 0 (SetVal):** The numeric value is right-aligned at the destination's LSD end. For Integer and Decimal, the value is further extended to fill the destination's width per §13.1.3.
- **CMB-4 = 1 (SetTxt):** The text value is left-aligned in the destination register, with the first characters at the high end.

13.1.2 Operand and value field

- **Operand 1 (dR1):** Destination register.
- **Operands 2-5:** 50-bit payload field (right-justified in the 60-bit span; bits 41–50 are unused padding and must be 0).

The destination register is in Operand 1, and the immediate payload occupies Operands 2 through 5. This is a special-form exception to the standard destination-last convention.

The 50-bit payload width imposes a hard limit on how much data a single **SetVal** or **SetTxt** instruction can embed directly.

13.1.3 Behavior

- **SetVal** loads a numeric payload into the destination register. When the destination is wider than 50 bits, the CPU applies type-aware extension based on CMB 1-2:
 - **CMB 1-2 = 10 (Integer):** sign-extends the 50-bit Int per Kiss reversed-sign convention — the payload's sign bit (bit 49) moves to the destination's MSB, bits 49 through (LenBits–2) are filled with **NOT(sign bit)**, and magnitude bits 0–48 are preserved. Without this, the 50-bit form's sign at bit 49 wouldn't reach the position where Integer arithmetic looks for it (the destination's MSB).

- **CMB 1-2 = 11 (Decimal):** right-aligns the encoded digits. The assembler packs N digits MSB-first at the top of the 50-bit payload with Null padding in the low slots; the CPU counts trailing Null digits and shifts the payload right by $(\text{trailing_nulls} \times 5)$ bits so the N actual digits land at the destination's LSD-most N positions per the standard right-aligned digit storage convention. The decimal-point marker (**DecPt**) is treated as a non-Null digit and travels with the data. *Single-digit example:* **SetVal R, #D:5** on a one-byte (10-bit) destination produces a byte with **00000** in the high half (bits 1-5, the MSD slot) and the digit code **10101 (+5)** in the low half (bits 6-10, the LSD slot).
- **CMB 1-2 = 01 (Binary) or 00 (untyped):** no extension; the 50-bit payload is written to the destination's low 50 bits and high bits remain zero. Binary has no sign concept; untyped is opaque.
- **SetTxt** loads a text payload into the destination register, left-aligned regardless of CMB 1-2 — the first characters land at the high end.

For destinations of 50 bits or narrower, no extension is applied. Integer values narrower than 50 bits truncate the magnitude to fit while preserving the sign bit at the destination's MSB.

This type-aware behavior landed in 2026-05-08 (Integer) and 2026-05-09 (Decimal) — see [docs/decisions.md](#) 2026-05-09 entry.

13.1.4 Assembler and compiler lowering

In source form, the programmer supplies an immediate value and a destination register. The assembler or compiler performs the following steps:

1. Converts the immediate to the intended type.
2. Places the destination register identifier into Operand 1 (dR1).
3. Packs the converted value into the 50-bit payload field in Operands 2-5 (right-justified; bits 41-50 zero-padded).
4. Sets CMB-4 = **0** (SetVal) or CMB-4 = **1** (SetTxt).

See section 3.4 for general immediate-value and datatype conventions.

13.1.5 Design note

At the assembler level, **SetVal** and **SetTxt** are two readable source forms that both lower to the single ISA command **SetImm**, differentiated by CMB-4.

13.2 Unimplemented and non-core commands

The following commands appear in assembler source, ISA command lists, or design documents but are **not** part of the active Kiss ISA. They are documented here for completeness and to prevent confusion.

13.2.1 Simulator and VM helpers

These entries exist to support the Kiss VM/Simulator environment and are not real hardware ISA commands:

Command	Description
A.* family (e.g., A.Add, A.Sub, A.Mul)	Kiss VM/Simulator helper commands. The A. prefix denotes simulator-internal operations that do not correspond to hardware instructions.
EOP (End of Program)	Signals program termination to the simulator. Likely a VM/simulator convention rather than a real hardware ISA command.
Else	An assembler/compiler structural keyword only. It directs control flow during assembly but does not emit a runtime ISA instruction.

13.2.2 Not implemented / planned

These commands have been discussed or partially designed but are not yet implemented in the ISA:

Command	Status
Except (Excpt)	Replaced by SysReq (voluntary app→kernel signaling) and the hardware fault architecture (involuntary). See the ISA reference for full SysReq and Fault Architecture specifications.
Sav, SavAll, Rst, RstAll	Design ideas for context save/restore. Parked for possible later addition — with dedicated kernel cores that may directly access thread register files, app-side save/restore instructions may not be needed.

14. Reference tables

This section consolidates key encodings and command-family summaries into quick-reference form. For full definitions and encoding rationale, consult the sections cited in each table.

14.1 Datatype codes

Standard two-bit datatype encoding used in CMB 1-2 (primary datatype) and CMB 7-8 (conversion target or immediate-value datatype, depending on family).

CMB Value	Datatype
00	Untyped / not used
01	Binary
10	Integer
11	Decimal

See section 3.4 for full immediate-value and datatype conventions.

14.2 Condition codes

The Jump, Call, and If command families share a common set of 30 condition suffixes. See section 3.5 for complete definitions.

#	Suffix	Condition	#	Suffix	Condition
1	Z	Zero	16	NNeg	Not negative
2	NZ	Not zero	17	EvN	Even
3	Blw	Below	18	Odd	Odd
4	BlwE	Below or equal	19	Ovr	Overflow
5	LT	Less than	20	NOvr	No overflow
6	LE	Less than or equal	21	Und	Underflow
7	EQ	Equal	22	NUnd	No underflow
8	NE	Not equal	23	Brw	Borrow
9	GE	Greater than or equal	24	NBrw	No borrow
10	GT	Greater than	25	Cry	Carry
11	AbvE	Above or equal	26	NCry	No carry
12	Abv	Above	27	THi	High-end truncation
13	Pos	Positive	28	NTHi	No high-end truncation
14	NPos	Not positive	29	TLo	Low-end truncation
15	Neg	Negative	30	NTLo	No low-end truncation

Note: **IfIRQ** is specific to the If command family and is not part of the shared condition-suffix set.

14.3 Family quick reference

Summary of every command family, its ISA commands, and CMB usage. Refer to the listed section for full encoding details.

Section	Family	ISA Command(s)	Key CMB Usage
4	Control and conditional-execution	NOP, Brk, Retrn, If*, EndIf, ClrCF	All zero (most); family-specific where needed
5	Jump	Jmp	1–2: imm type, 3: imm ind, 4: PC-rel, 5: backward, 6–8: CEB clear count
6	Call	Call	1–2: imm type, 3: imm ind, 4: PC-rel, 5: backward, 6–8: unused/zero
7.2	Load / Store	Load, Stor	4: direction, 5: offset enable, 6: multiplier enable
7.3	Push / Pull	Push, Pull	3: imm ind, 7–8: imm type
7.4	Copy / Move	Copy, Move	1–2: datatype, 4: direction
7.5	Read / Write (I/O)	Read, Write	4: direction
7.6	Register utilities	ClrReg, Swap	All zero
8.2	Ordinary arithmetic	Add, Sub, Mul, Div	1–2: datatype, 3: imm ind, 4: saturation, 5: carry/borrow, 6: set flags
8.3	Fused multiply-add	MulAdd	Same as §8.2
8.4	Divide with remainder	DivRem	1–2: datatype, 3: imm ind, 4: saturation, 5: DivRem indicator, 6: set flags
8.5	Apply-to	AddTo, SubTo, MulTo, DivTo	Kiss-50: 1–2 datatype, 4 Sat, 5 Opr1 immediate value selector

Section	Family	ISA Command(s)	Key CMB Usage
8.6	Increment / Decrement	Incr, Decr	Kiss-50: 1-2 datatype, 3 SetFlags, 5 Opr1 immediate amount selector
9	Rotate / Shift	Rot, Shift, SShift	1-2: datatype, 3: imm ind, 4: direction, 5: unused (=0), 6: set flags, 7-8: imm type
10.3	Ordinary logic	And, AndN, NAnd, Or, OrN, NOOr, XOr, NXOr	1-2: datatype, 6: set flags
10.4	Not	Not	1-2: datatype, 6: set flags
10.5	Neg	Neg	1-2: datatype, 6: set flags
10.6	BitFlip	BitFlip	Same as §10.4
10.7	Single-bit set/test	BitSet, BitTst	3: position imm, 4: on(1)/off(0)
11	Test / Compare / Bounds	Test, Compare, ChkBnds	1-2: datatype, 6: set flags
12	Conversion	Cvt	1-2: source type, 7-8: dest type
13.1	SetVal / SetTxt	SetVal, SetTxt	3: imm ind (always 1), 4: val/txt selector

15. Examples

This section shows how source-level assembler forms lower into the ISA instruction encoding. Each example gives the assembler source line, the field-by-field lowering, and a short note on how the encoding follows the family's general rules.

15.1 Example 1 — Typed arithmetic with flag update

Assembler Source: `I.AddSet R1WA R1WB R1WC`

Add Integer R1WA + R1WB, write the result to R1WC, and update the arithmetic status flags.

Lowering:

Field	Value	Meaning
CEB	00000	unconditional
Command	Add (opcode 704)	base arithmetic command
CMB	1 0 0 0 0 1 0 0	CMB 1-2 = 10 Integer; CMB-6 = 1 Set Status Flags; all other CMB bits 0
Operand 1	R1WA (16)	first source (sR1)
Operand 2	R1WB (17)	second source (sR2)
Operand 3	0	unused
Operand 4	0	unused
Operand 5	R1WC (18)	destination (dR5)

Pattern: `sR1 sR2 dR5` — the destination-last convention applies. The `Set` suffix sets CMB-6 so that the Result-property group (Z, Pos, Neg, EVN, ODD) and the Arithmetic group (OVR / UND / WRP / CRY as appropriate to the outcome) are updated after the Add. Compare-group flags are preserved across the Add.

15.2 Example 2 — Load with offset (Pattern B data movement)

Assembler Source: `LoadLOfs R1WA R2WA R3WA R4WA`

Load bytes from memory at `R2WA + R3WA` into `R4WA`. Transfer byte count is given by `R1WA`; alignment is left-to-right.

Lowering:

Field	Value	Meaning
CEB	00000	unconditional
Command	Load (opcode 3020)	base load command
CMB	0 0 0 1 1 0 0 0	CMB-4 = 1 left alignment; CMB-5 = 1 offset enable; CMB-6 = 0 no multiplier; all other CMB bits 0
Operand 1	R1WA	byte count (iR1)
Operand 2	R2WA	base memory address (asR2)
Operand 3	R3WA	offset (sR3)
Operand 4	0	unused (no multiplier)
Operand 5	R4WA	destination register (dR5)

Pattern: `iR1 asR2 sR3 dR5` — Load family. Because CMB-5 = 1 and CMB-6 = 0, the effective address is `baseAddr + offset`. Direction-explicit assembler form `LoadLOfs` maps to CMB-4 = 1 (left-aligned).

15.3 Example 3 — SetImm payload form (special-form exception)

Assembler Source: `SetVal R1WA #B:12345`

Load the literal `12345` into `R1WA`. The `#B:` prefix tells the **assembler** to encode the payload as an unsigned binary value; this type information is consumed entirely at assembly time and does not appear in the ISA instruction.

Lowering:

Field	Value	Meaning
CEB	00000	unconditional
Command	SetImm (opcode 550)	immediate-load base command
CMB	0 0 1 0 0 0 0 0	CMB-3 = 1 immediate-form indicator; CMB-4 = 0 numeric (SetVal, not SetTxt); all other CMB bits 0
Operand 1	R1WA (16)	destination register (dR1) — special-form exception to destination-last
Operand 2 (upper, bits 41–50)	0000000000	unused padding (must be 0)
Operand 2 (lower) + Operands 3–5 (bits 51–100)	payload 12345 right-justified in 50 bits	immediate value (as encoded by the assembler)

Pattern: `SetImm` is the only ISA command that writes to Operand 1 rather than Operand 5. The 50-bit payload occupies bits 51–100 of the instruction; bits 41–50 are unused padding. For a

value of 12345 (**0b11000000111001**), the payload's low 14 bits carry the value and the upper 36 bits are zero-padded. CMB 1-2 carries the data type (**01** Binary in this example), which the CPU uses to apply type-aware extension when the destination register is wider than 50 bits — see §13.1.3 for the per-type extension rules. For Binary and untyped payloads the extension is a no-op (raw write); Integer payloads sign-extend per Kiss reversed-sign convention; Decimal payloads right-align their digits at the destination's LSD end.

16. Instruction format reference by family

This section presents each command family as a table of ISA commands showing their instruction encoding and a brief description. Each row represents one ISA command; the CEB field is omitted because it is always **00000** for an unconditional instance (CEB state is discussed in section 3.2).

Letter placeholders in the CMB column are fields whose bit values vary among assembler variants (data type, Set flag, etc.); their encoding is given in a per-family legend below each table. Operand notation uses the abbreviations defined in section 2.2 (**sR**, **dR**, **sdR**, **iR**, **asR**, **adR**, **osR**, **odR**, **osdR**). A — marks an unused operand slot, which must be **0**.

16.1 Execution control

Commands: **NOP**, **Brk**, **Retrn**, **ClrCF**. All use no operands and all eight CMB bits are **0**.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
NOP	00000000	—	—	—	—	—	No operation (pipeline placeholder)
Brk	00000000	—	—	—	—	—	Breakpoint / debugger trap
Retrn	00000000	—	—	—	—	—	Return from subroutine; restore saved state from shadow call stack
ClrCF	00000000	—	—	—	—	—	Reset CEB state (Main = Shadow = 00000, Depth = 0)

16.2 Conditional block commands

The **If*** family opens conditional blocks and **EndIf** closes them. All variants use no operands and all eight CMB bits are **0**; the condition is encoded entirely in the Command field (distinct opcode per variant). A few representative rows:

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
IfIRQ	00000000	—	—	—	—	—	Open If block if interrupt-request pending
IfZ	00000000	—	—	—	—	—	Open If block if Z flag set
IfGT	00000000	—	—	—	—	—	Open If block if signed greater-than (GT flag)
IfNCry	00000000	—	—	—	—	—	Open If block if Carry clear
EndIf	00000000	—	—	—	—	—	Close one active If nesting level

The complete set is 33 **If*** variants plus **EndIf**, listed in §4.3.1. Every variant shares the same all-

zero CMB and empty operand list shown above.

16.3 Jump commands

Jump commands transfer execution to a target instruction location. Each conditional variant is a distinct ISA command; each command supports three direction modes (absolute, forward-relative, backward-relative) via CMB-4 / CMB-5. A few representative rows:

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Jmp	JJI00CCC	iR1	—	—	—	—	Unconditional absolute jump; PC = Opr1
FJmp	JJI10CCC	iR1	—	—	—	—	Unconditional forward-relative jump; PC = PC + Opr1
BJmp	JJI11CCC	iR1	—	—	—	—	Unconditional backward-relative jump; PC = PC – Opr1
JmpZ	JJI00CCC	iR1	—	—	—	—	Jump absolute if Z flag set
JmpGT	JJI00CCC	iR1	—	—	—	—	Jump absolute if signed greater-than
FJmpEQ	JJI10CCC	iR1	—	—	—	—	Forward-relative jump if equal
BJmpLE	JJI11CCC	iR1	—	—	—	—	Backward-relative jump if signed less-than-or-equal
JmpNCry	JJI00CCC	iR1	—	—	—	—	Jump absolute if Carry clear

The complete set is 31 base variants (unconditional **Jmp** + 30 conditional **Jmp***), each available in absolute, forward-relative (**FJmp***), and backward-relative (**BJmp***) direction forms selected via CMB-4 / CMB-5. See §3.5 for the full condition-suffix list.

Legend (Jump family):

- **JJ** (CMB 1-2) = immediate-value data type, used when **I** = **1** (same encoding as in §3.4).
- **I** (CMB-3) = Operand-1 source kind: **0** register ID, **1** immediate value.
- CMB-4 = PC-relative mode: **0** absolute, **1** PC-relative.
- CMB-5 = backward-relative selector (meaningful only when CMB-4 = **1**): **0** forward, **1** backward.
- **CCC** (CMB 6-8) = CEB clear count — how many active conditional-execution levels the jump unwinds when taken.

16.4 Call commands

Call commands invoke a subroutine. Like Jump commands, each conditional variant is a distinct ISA command with three direction forms, but CMB 6–8 are unused because a Call performs a full protected-state save rather than a partial CEB unwind. A few representative rows:

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Call	JJI00000	iR1	—	—	—	—	Unconditional absolute call; save state, branch to Opr1

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
FCall	JJI10000	iR1	—	—	—	—	Unconditional forward-relative call; target = PC + Opr1
BCall	JJI11000	iR1	—	—	—	—	Unconditional backward-relative call; target = PC – Opr1
CallZ	JJI00000	iR1	—	—	—	—	Call absolute if Z flag set
CallGT	JJI00000	iR1	—	—	—	—	Call absolute if signed greater-than
FCallEQ	JJI10000	iR1	—	—	—	—	Forward-relative call if equal
BCallLE	JJI11000	iR1	—	—	—	—	Backward-relative call if signed less-than-or-equal
CallNCry	JJI00000	iR1	—	—	—	—	Call absolute if Carry clear

The complete set is 31 base variants (unconditional **Call** + 30 conditional **Call***), each available in absolute, **FCall***, and **BCall*** direction forms. A Call saves R1-R4, the Status Flag register, CEB state, and special-register state to the shadow call stack, then enters the called routine with CEB = **00000**. **Retrn** (§16.1) restores the saved state.

Legend (Call family):

- **JJ** (CMB 1-2) = immediate-value data type, used when **I** = **1**.
- **I** (CMB-3) = Operand-1 source kind: **0** register, **1** immediate.
- CMB-4 = PC-relative mode: **0** absolute, **1** PC-relative.
- CMB-5 = backward-relative selector (meaningful only when CMB-4 = **1**).
- CMB 6-8 = unused, must be **0**.

16.5 Load and Store

Load and Store transfer data between memory and registers. Three assembler source forms per direction exist, distinguished by CMB-5 (offset enable) and CMB-6 (multiplier enable). Direction (left / right alignment) is controlled by CMB-4.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Load	000D0000	iR1	asR 2	—	—	dR5	R5 ← mem[R2], Opr1 bytes (base only)
Load	000D1000	iR1	asR 2	sR3	—	dR5	R5 ← mem[R2 + R3], Opr1 bytes
Load	000D1100	iR1	asR 2	sR3	sR4	dR5	R5 ← mem[R2 + R3 × R4], Opr1 bytes
Stor	000D0000	iR1	sR2	adR 3	—	—	mem[R3] ← R2, Opr1 bytes (base only)
Stor	000D1000	iR1	sR2	adR 3	sR4	—	mem[R3 + R4] ← R2, Opr1 bytes
Stor	000D1100	iR1	sR2	adR	sR4	sR5	mem[R3 + R4 × R5] ← R2, Opr1 bytes

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
				3			

Legend:

- **D** (CMB-4) = alignment direction: **0** right-to-left (***R** suffix), **1** left-to-right (***L** suffix).
- CMB-5 = offset enable (**1** adds an offset operand).
- CMB-6 = multiplier enable (**1** adds a multiplier operand, valid only when CMB-5 = **1**).

The assembler exposes **LoadL** / **LoadR**, **LoadLOfs** / **LoadROfs**, **LoadLOfsMul** / **LoadROfsMul**, and the matching **Stor*** forms; each maps to the **Load** or **Stor** ISA command with the appropriate CMB bits.

16.6 Push and Pull

Push and Pull transfer data between registers and one of ten data stacks. Operand 1 is the stack selector (1-10); operands 2-5 are the registers to push or pull in order. Up to four registers can be transferred in a single instruction.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Push	00I000JJ	iR1	sR2	osR3	osR4	osR5	push R2 (...R5) onto stack R1 with trailing size byte
Pull	00I000JJ	iR1	dR2	odR3	odR4	odR5	pull from stack R1 into R2 (...R5)

Legend:

- **I** (CMB-3) = Operand-1 source kind: **0** stack selector is a register, **1** immediate stack ID.
- **JJ** (CMB 7-8) = immediate data type for the stack selector (used when **I** = **1**).

16.7 Copy and Move

Copy duplicates a source register value into one or more destination registers; the source is unchanged. Move is identical except the source is zeroed after all destinations have been written.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Copy	TT0D0000	sR1	odR2	odR3	odR4	dR5	R5 ← R1 (+ fanout to R2, R3, R4 if non-zero)
Move	TT0D0000	sR1	odR2	odR3	odR4	dR5	R5 ← R1; then R1 ← 0 (+ fanout to R2, R3, R4)

Legend:

- **TT** (CMB 1-2) = data type: **01** Binary, **10** Integer, **11** Decimal. Controls how the value is extended or truncated when copied across different register widths.

- **D** (CMB-4) = alignment direction: **0** right, **1** left.

16.8 Read and Write (I/O)

Read and Write transfer bytes between registers and hardware I/O ports. They are type-agnostic: the 50-bit payload is moved verbatim, with CMB-4 controlling alignment.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Read	00ID0000	iR1	sR2	—	—	dR5	$R5 \leftarrow \text{port}[R2]$, Opr1 bytes
Write	00ID0000	iR1	sR2	sR3	—	—	$\text{port}[R2] \leftarrow R3$, Opr1 bytes

Legend:

- **I** (CMB-3) = Operand-1 selector: **0** byte count is a register, **1** byte count is immediate.
- **D** (CMB-4) = alignment direction: **0** right-to-left (***R**), **1** left-to-right (***L**).

16.9 Register utilities

ClrReg zeros one or more registers. **Swap** exchanges the contents of two registers. All CMB bits are **0** for both.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
ClrReg	00000000	odR 1	odR 2	odR 3	odR 4	dR5	$R5 \leftarrow 0$ (+ fanout: any non-zero Opr1-Opr4 also cleared)
Swap	00000000	sdR 1	—	—	—	sdR 5	exchange R1 $\leftrightarrow$ R5

16.10 Ordinary arithmetic (Add, Sub, Mul, Div)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Add	TTIACSJJ	sR1	sR2	osR 3	osR 4	dR5	$R5 = R1 + R2 (+ R3 + R4)$
Sub	TTIACSJJ	sR1	sR2	osR 3	osR 4	dR5	$R5 = R1 - R2 (- R3 - R4)$
Mul	TTIA0SJJ	sR1	sR2	osR 3	osR 4	dR5	$R5 = R1 \times R2 (\times R3 \times R4)$
Div	TTIA0SJJ	sR1	sR2	osR 3	osR 4	dR5	$R5 = R1 \div R2 (\div R3 \div R4)$

Legend:

- **TT** (CMB 1-2) = data type: **01** Binary, **10** Integer, **11** Decimal.
- **I** (CMB-3) = immediate indicator.
- **A** (CMB-4) = saturation: **0** wrap, **1** saturate.

- **C** (CMB-5) = carry (Add) / borrow (Sub); must be **0** for Mul and Div.
- **S** (CMB-6) = Set Status Flags.
- **JJ** (CMB 7-8) = immediate data type (used when **I** = **1**).

Optional operands **osR3** / **osR4** chain additional sources into the same operator. When absent, only R1 and R2 participate.

16.11 Fused multiply-add (MulAdd)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
MulAdd	TTIA0SJJ	sR1	sR2	sR3	—	dR5	$R5 = (R1 \times R2) + R3$

Legend: same as §16.10. CMB-5 is unused and must be **0**. **MulAdd** is a fixed 4-operand form — Opr4 is unused.

16.12 Divide with remainder (DivRem)

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
DivRem	TTIA1SJJ	sR1	sR2	—	dR4	dR5	$R5 = R1 \div R2$ (quotient); R4 = remainder

Legend: same as §16.10. CMB-5 = **1** distinguishes **DivRem** from plain **Div**. Opr3 is unused.

16.13 Apply-to families (AddTo, SubTo, MulTo, DivTo)

These “apply-to” forms take an amount in Opr1 and apply it in place to the primary target (Opr5) and any optional targets (Opr2-Opr4). Each target is read, modified, and written back.

Command	Kiss-50 CMB	Opr 1	Opr 2	Opr 3	Description
AddTo	TT0AI	iR1	sdR2	-	$R2 += \text{value}$
SubTo	TT0AI	iR1	sdR2	-	$R2 -= \text{value}$
MulTo	TT0AI	iR1	sdR2	-	$R2 \times = \text{value}$
DivTo	TT0AI	iR1	sdR2	-	$R2 \div = \text{value}$

Legend: **A** is saturation (CMB-4). **I** is the operand-1 immediate value selector (CMB-5): **0** = value register, **1** = compact immediate value. **Set**, **Cry**, and **Brw** are not valid for the Apply-to family.

16.14 Increment and Decrement (Incr, Decr)

Amount-driven in-place updates. In compact Kiss-50, CMB-5 selects whether operand 1 is a compact immediate amount (**1**) or an amount register (**0**). Incr / Decr do not support saturation (CMB-4 must be **0**) or carry/borrow.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Incr	Kiss-50: TTS0I	iR1	sdR2	-	-	-	$R2 += \text{amount}$; I/CMB-5 selects immediate amount vs register amount

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Decr	Kiss-50: TTS0I	iR1	sdR 2	-	-	-	R2 -= amount; I/CMB-5 selects immediate amount vs register amount

Legend for compact Kiss-50: **TT** = data type (CMB 1-2), **S** = SetFlags (CMB-3), CMB-4 is unused/must be **0**, and **I** = Opr1 immediate amount selector (CMB-5).

16.15 Rotate and Shift (Rot, Shift, SShift)

Count is in Opr1; the primary target is Opr5 and any non-zero Opr2-Opr4 are additional targets, all modified in place by the same count.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Rot	TTID0SJJ	iR1	osd R2	osd R3	osd R4	sdR 5	rotate R5 by R1 bits/digits (each target rotated in place; Decimal is digit-level)
Shift	TTID0SJJ	iR1	osd R2	osd R3	osd R4	sdR 5	logical shift R5 by R1 (each target shifted in place; Decimal is digit-level with sign-matched zero fill)
SShift	TTID0SJJ	iR1	osd R2	osd R3	osd R4	sdR 5	arithmetic (sign-preserving) shift R5 by R1 — Integer (TT = 10) only

Legend:

- **TT** (CMB 1-2) = data type.
- **I** (CMB-3) = immediate indicator for the count.
- **D** (CMB-4) = direction: **0** right, **1** left.
- CMB-5 is unused (must be **0**) — through-carry has been retired across the family.
- **S** (CMB-6) = Set Status Flags.
- **JJ** (CMB 7-8) = immediate data type.

16.16 Ordinary logic

Commands: **And**, **AndN**, **NAnd**, **Or**, **OrN**, **NOr**, **XOr**, **NXOr**.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
And	TT000S00	sR1	sR2	—	—	dR5	$R5 = R1 \wedge R2$
AndN	TT000S00	sR1	sR2	—	—	dR5	$R5 = R1 \wedge \neg R2$
NAnd	TT000S00	sR1	sR2	—	—	dR5	$R5 = \neg(R1 \wedge R2)$
Or	TT000S00	sR1	sR2	—	—	dR5	$R5 = R1 \vee R2$
OrN	TT000S00	sR1	sR2	—	—	dR5	$R5 = R1 \vee \neg R2$
NOr	TT000S00	sR1	sR2	—	—	dR5	$R5 = \neg(R1 \vee R2)$
XOr	TT000S00	sR1	sR2	—	—	dR5	$R5 = R1 \oplus R2$

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
NXOr	TT000S00	sR1	sR2	—	—	dR5	$R5 = \neg(R1 \oplus R2)$

Legend:

- **TT** (CMB 1-2) = data type: **01** Binary, **10** Integer, **11** Decimal.
- **S** (CMB-6) = Set Status Flags.

16.17 Not

Not performs bitwise inversion in place. Up to five registers may be inverted in one instruction when **S** = **0**; when **S** = **1** (Set Status Flags), only the primary target Opr5 is permitted so the flags reflect a single deterministic result.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Not	TT000S00	osd R1	osd R2	osd R3	osd R4	sdR 5	$R5 = \neg R5$; each non-zero target also inverted in place

Legend: same as §16.16.

16.18 Neg

Neg performs arithmetic sign inversion in place on a single register. Binary is not a valid data type (unsigned values have no arithmetic negation); CMB 1-2 must be **10** Integer or **11** Decimal.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Neg	TT000S00	—	—	—	—	sdR 5	$R5 = -R5$ (sign inverted in place)

Legend: **TT** = **10** Integer or **11** Decimal; **S** = Set Status Flags. No fanout.

16.19 BitFlip

Functionally identical to **Not** — exists as a separate command in the bit-processing family for semantic clarity.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
BitFlip	TT000S00	osd R1	osd R2	osd R3	osd R4	sdR 5	R5 bits inverted in place (same as Not)

Legend: same as §16.16.

16.20 Single-bit set / test

BitSet forces a specific bit on or off; **BitTst** reads a specific bit's value and sets the Z flag accordingly. CMB-4 selects whether the operation acts on **1** (on) or **0** (off).

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
BitSet (on)	00I10000	iR1	—	—	—	sdR 5	set bit at position Opr1 in R5 to 1
BitSet (off)	00I00000	iR1	—	—	—	sdR 5	set bit at position Opr1 in R5 to 0
BitTst (on)	00I10100	iR1	—	—	—	sR5	test bit Opr1 in R5; Z = 1 if that bit is 1
BitTst (off)	00I00100	iR1	—	—	—	sR5	test bit Opr1 in R5; Z = 1 if that bit is 0

Legend:

- **I** (CMB-3) = Opr1 form: **0** register holds bit position, **1** bit position is immediate.
- CMB-4 = on (**1**) / off (**0**) selector.
- CMB-6 = **1** for **BitTst** (must update the Z flag); **0** for **BitSet**.

Bit positions are 1-based (bit 1 = least significant). Maximum position = 2000 (range of the widest register bank).

16.21 Bit field operations (BitRead, BitWrite, BitClear)

Operate on a contiguous range of bits starting at a specified position with a specified width.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
BitRead	00PW0S00	iR1	iR2	sR3	dR4	—	R4 = bits [Opr1 : Opr1+Opr2–1] of R3, low-aligned
BitWrite	00PW0S00	iR1	iR2	sR3	sdR 4	—	insert low Opr2 bits of R3 into R4 starting at bit Opr1
BitClear	00PW0S00	iR1	iR2	sdR 3	—	—	clear bits [Opr1 : Opr1+Opr2–1] of R3

Legend:

- **P** (CMB-3) = position operand form: **0** register, **1** immediate.
- **W** (CMB-4) = width operand form: **0** register, **1** immediate.
- **S** (CMB-6) = Set Status Flags.

16.22 Bit counting and finding

Count or locate set / clear bits within a register. All variants share the same operand layout (**sR1**, **dR5**); CMB-3 and CMB-4 select the variant.

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
BitCntTot (on)	00010S00	sR1	—	—	—	dR5	R5 = number of 1 bits in R1
BitCntTot (off)	00000S00	sR1	—	—	—	dR5	R5 = number of 0 bits in R1
BitCnt (lead on)	00110S00	sR1	—	—	—	dR5	R5 = count of leading 1 bits in R1

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
BitCnt (lead off)	00100S00	sR1	—	—	—	dR5	R5 = count of leading 0 bits in R1
BitCnt (trail on)	00010S00	sR1	—	—	—	dR5	R5 = count of trailing 1 bits in R1
BitCnt (trail off)	00000S00	sR1	—	—	—	dR5	R5 = count of trailing 0 bits in R1
BitFind (first)	00110S00	sR1	—	—	—	dR5	R5 = position of first 1 bit from left (0 if none; Z set when none)
BitFind (last)	00010S00	sR1	—	—	—	dR5	R5 = position of last 1 bit from right (0 if none; Z set when none)

Legend varies by ISA command (**BitCntTot** uses CMB-4 alone; **BitCnt** uses CMB-3 for lead/trail + CMB-4 for on/off; **BitFind** uses CMB-3 for first/last with CMB-4 = **1**). See §10.9 for the full table of assembler mnemonics and exact CMB values.

16.23 Bit and byte reversal

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
BitRev	00000S00	sR1	—	—	—	dR5	R5 = R1 with bit order reversed
ByteRev	00000S00	sR1	—	—	—	dR5	R5 = R1 with byte (10-bit unit) order reversed

Legend: **S** (CMB-6) = Set Status Flags.

16.24 Test, Compare, and Bounds

These commands evaluate operands and set status flags. CMB-6 is **required** to be **1** for every instance (the family exists solely to set flags).

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Test	TT000100	sR1	—	—	—	—	compare R1 against zero; set arithmetic flags
Compare	TT000100	sR1	sR2	—	—	—	compare R1 vs R2; set Compare-group flags (EQ, GT, GE, LT, LE, Abv, AbvE, Blw, BlwE)
ChkBnds	TT000100	sR1	sR2	sR3	—	—	test whether $R2 \leq R1 \leq R3$; set compare flags

Legend:

- **TT** (CMB 1-2) = data type: **01** Binary, **10** Integer, **11** Decimal.
- CMB-6 must be **1**.

16.25 Conversion (Cvt)

Cvt converts a value from one data type to another. Unique among ISA commands in using CMB 1-2 and CMB 7-8 for two different datatype roles (source and destination).

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
Cvt	SS0000DD	sR1	—	—	—	dR5	R5 = R1 converted from source type (SS) to

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
							destination type (DD)

Legend:

- **SS** (CMB 1-2) = source data type: **01** Binary, **10** Integer, **11** Decimal.
- **DD** (CMB 7-8) = destination data type: same encoding.

The assembler spells conversions with a dotted source-dest prefix: **B.I.Cvt**, **I.D.Cvt**, **D.B.Cvt**, etc.

16.26 SetVal and SetTxt (SetImm)

Special-form command: the destination register is in Operand 1 rather than Operand 5, and the 50-bit immediate payload is packed across Operands 2-5 (right-justified; bits 41-50 unused padding).

Command	CMB	Opr 1	Opr 2	Opr 3	Opr 4	Opr 5	Description
SetVal (SetImm numeric)	TT100000	dR1	imm	imm	imm	imm	R1 ← 50-bit numeric payload, type-aware extension per CMB 1-2 (§13.1.3)
SetTxt (SetImm text)	00110000	dR1	imm	imm	imm	imm	R1 ← 50-bit text payload, left-aligned

Legend: **TT** (CMB 1-2) = data type — **00** untyped, **01** Binary, **10** Integer, **11** Decimal. The CPU uses CMB 1-2 to apply type-aware extension when the destination is wider than 50 bits (Integer sign-extension; Decimal digit right-alignment; Binary/untyped raw write). SetTxt always uses CMB 1-2 = **00** (text payload is left-aligned regardless).

Payload packing across Opr 2-5: bits 41-50 are unused padding, bits 51-55 hold payload bits 1-5 (low end of Opr 2), Opr 3 holds payload bits 6-20, Opr 4 holds bits 21-35, Opr 5 holds bits 36-50.

Additional CMB notes:

- CMB-3 always **1** (immediate-form indicator).
- CMB-4 = **0** numeric (SetVal, right-aligned) / **1** text (SetTxt, left-aligned).
- CMB 5-8 are unused and must be **0**.